

Deep Learning

Deep Learning for Text



Prof. Anxiao (Andrew) Jiang

Learning Objectives:

1. Understand sequence-modeling approaches for text
2. Understand the transformer architecture
3. Understand sequence-to-sequence learning



Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with Embedding layer (with masking enabled), Bi-direc LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-direc LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer encoder

1.7 Try 6: Transformer encoder with positional embedding

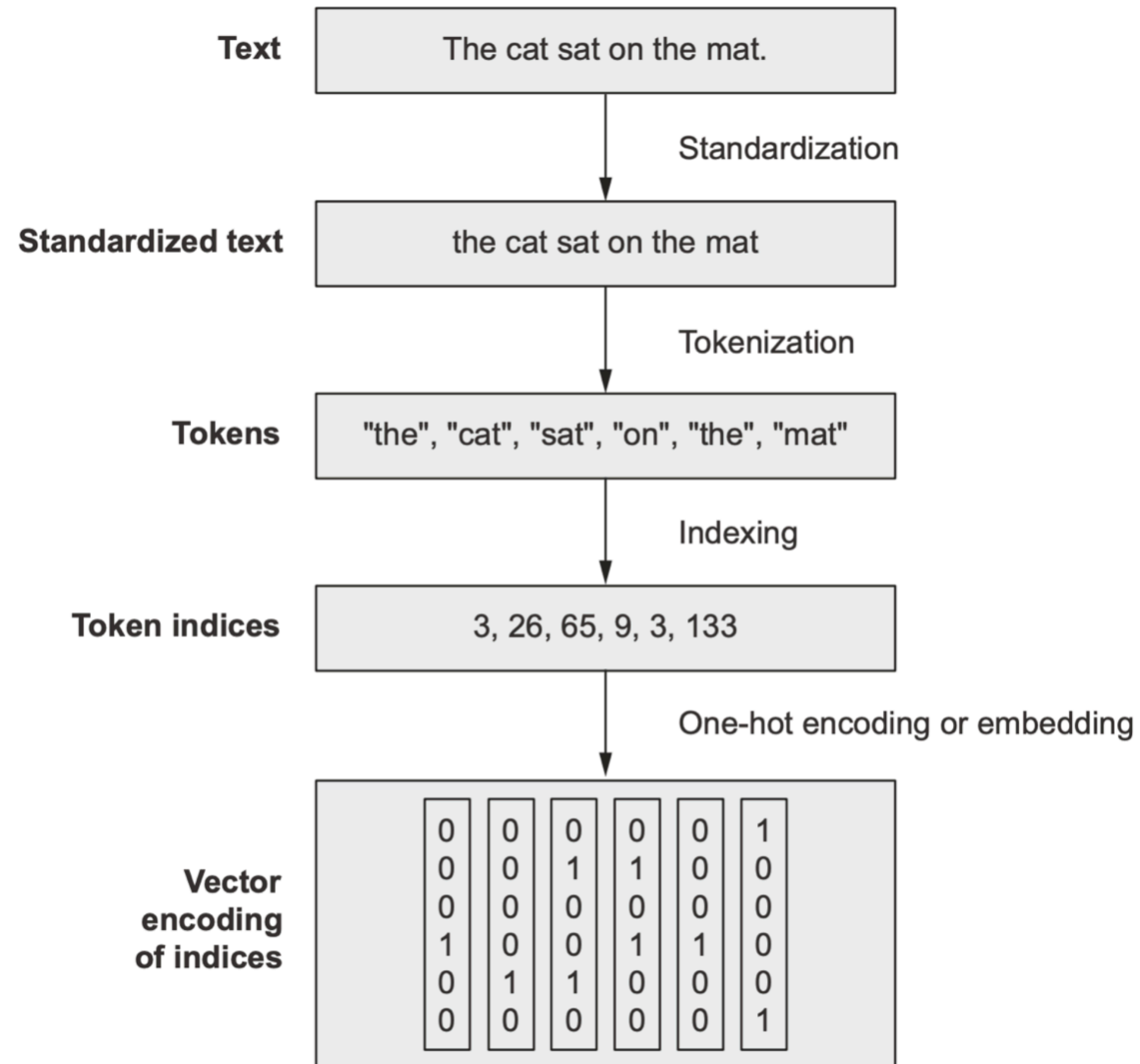
2. Sequence-to-Sequence Learning for Machine Translation



Topics in Natural Language Processing (NLP)

- “What’s the topic of this text?” (text classification)
- “Does this text contain abuse?” (content filtering)
- “Does this text sound positive or negative?” (sentiment analysis)
- “What should be the next word in this incomplete sentence?” (language modeling)
- “How would you say this in German?” (translation)
- “How would you summarize this article in one paragraph?” (summarization)
- etc.

Prepare and **Vectorize** Text



Listing 11.13 A sequence model built on one-hot encoded vector sequences

```
import tensorflow as tf
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = tf.one_hot(inputs, depth=max_tokens)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()
```

One input is a
sequence of integers.

Encode the integers
into binary 20,000-
dimensional vectors.

Add a
bidirectional
LSTM.

Finally, add a
classification
layer.



Test accuracy = 87%

Quiz question:

I. What are the pros and cons of using LSTM for text?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with Embedding layer (with masking enabled), Bi-direc LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-direc LSTM

1.5 Attention and Transformer

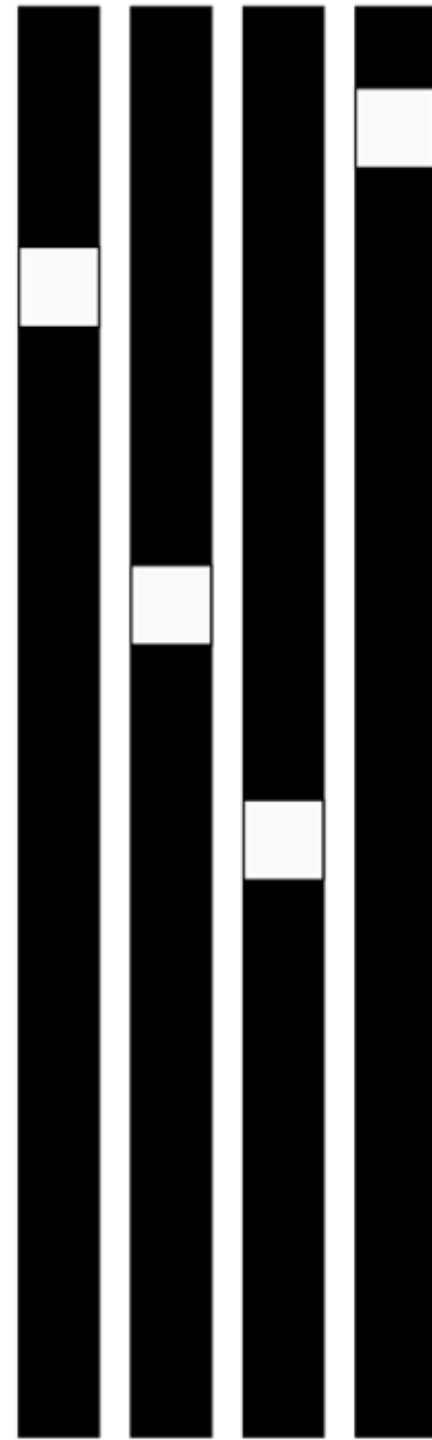
1.6 Try 5: Transformer encoder

1.7 Try 6: Transformer encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation

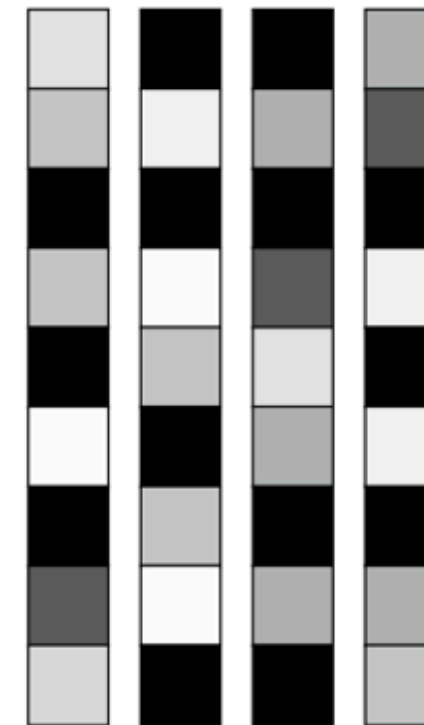


Use **word embedding**



One-hot word vectors:

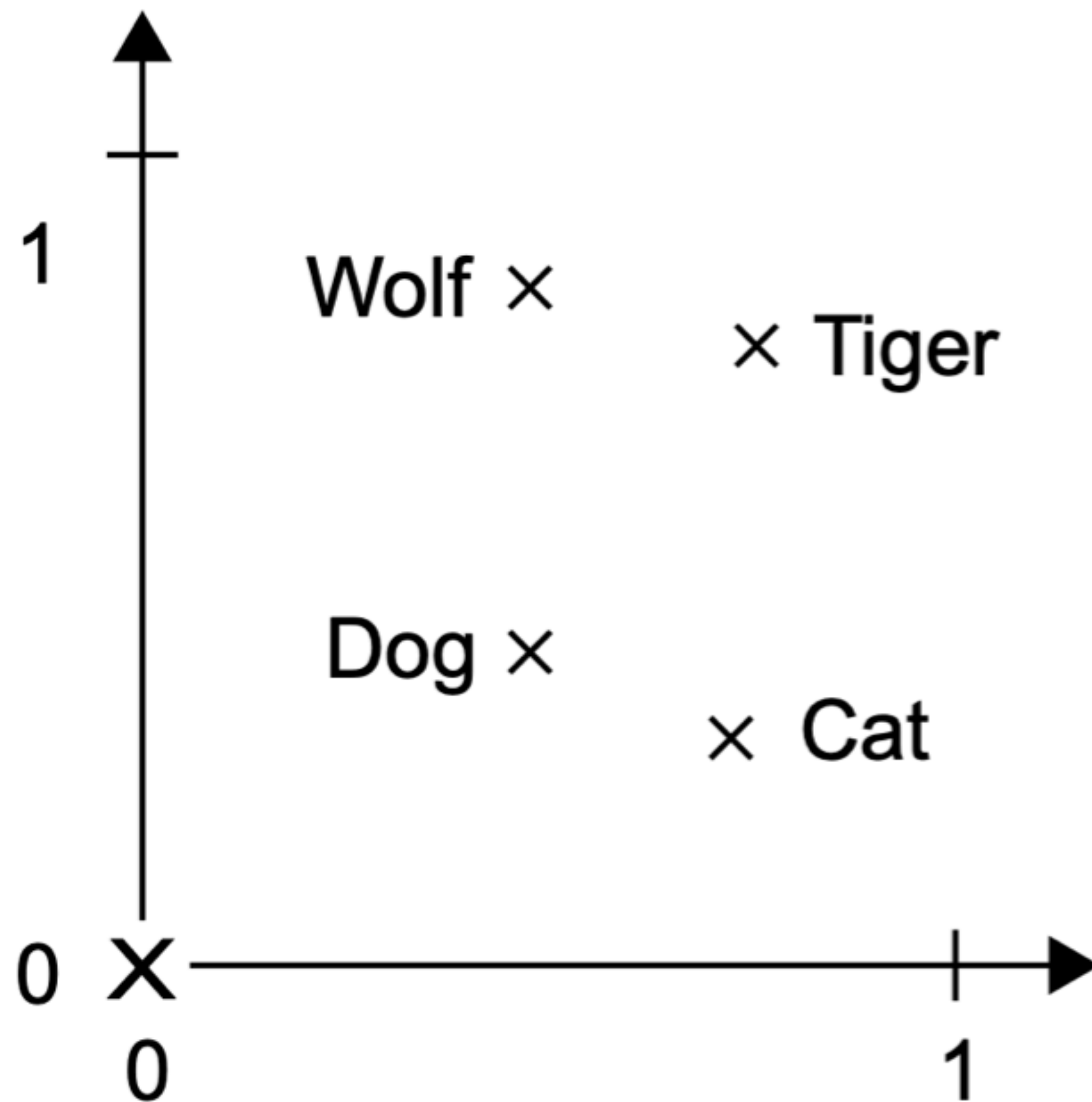
- Sparse
- High-dimensional
- Hardcoded



Word embeddings:

- Dense
- Lower-dimensional
- Learned from data

Structure revealed by word embedding



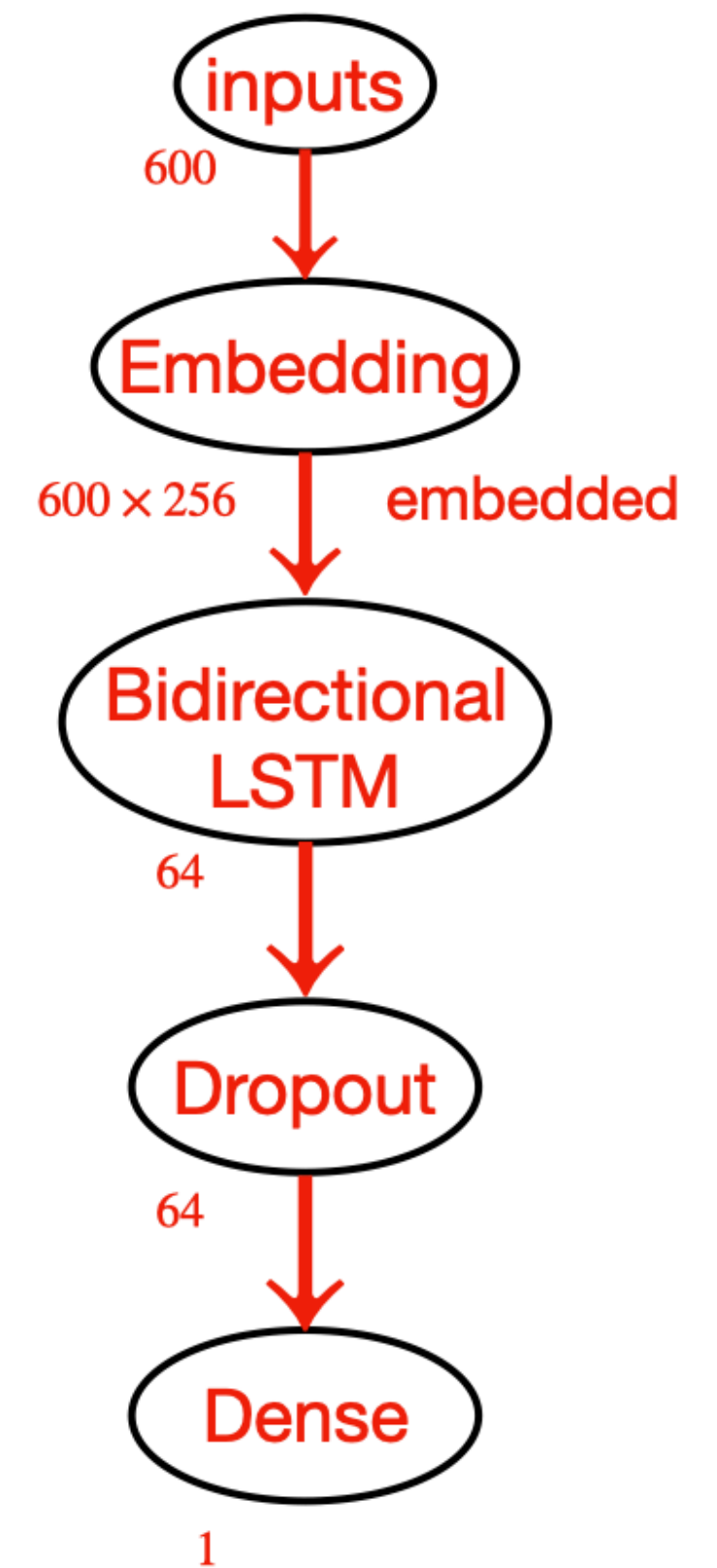
How to get word embedding

1. Learn word embedding jointly with the main task
2. Use *pre-trained word embeddings*.

Listing 11.16 Model that uses an Embedding layer trained from scratch

```
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(input_dim=max_tokens, output_dim=256)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("embeddings_bidir_gru.keras",
                                    save_best_only=True)
]
model.fit(int_train_ds, validation_data=int_val_ds, epochs=10,
        callbacks=callbacks)
model = keras.models.load_model("embeddings_bidir_gru.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```



Test accuracy = 87%

Quiz question:

- I. What is the benefit of word embedding, compared to one-hot encoding?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with the embedding layer (with masking enabled), Bi-directional LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-direc LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer encoder

1.7 Try 6: Transformer encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation



Understanding **Padding** and **Masking**

Issue of previous approach: shorter sequences are **padded** with 0s at the end.

The information remembered by RNN can fade away with those 0s.

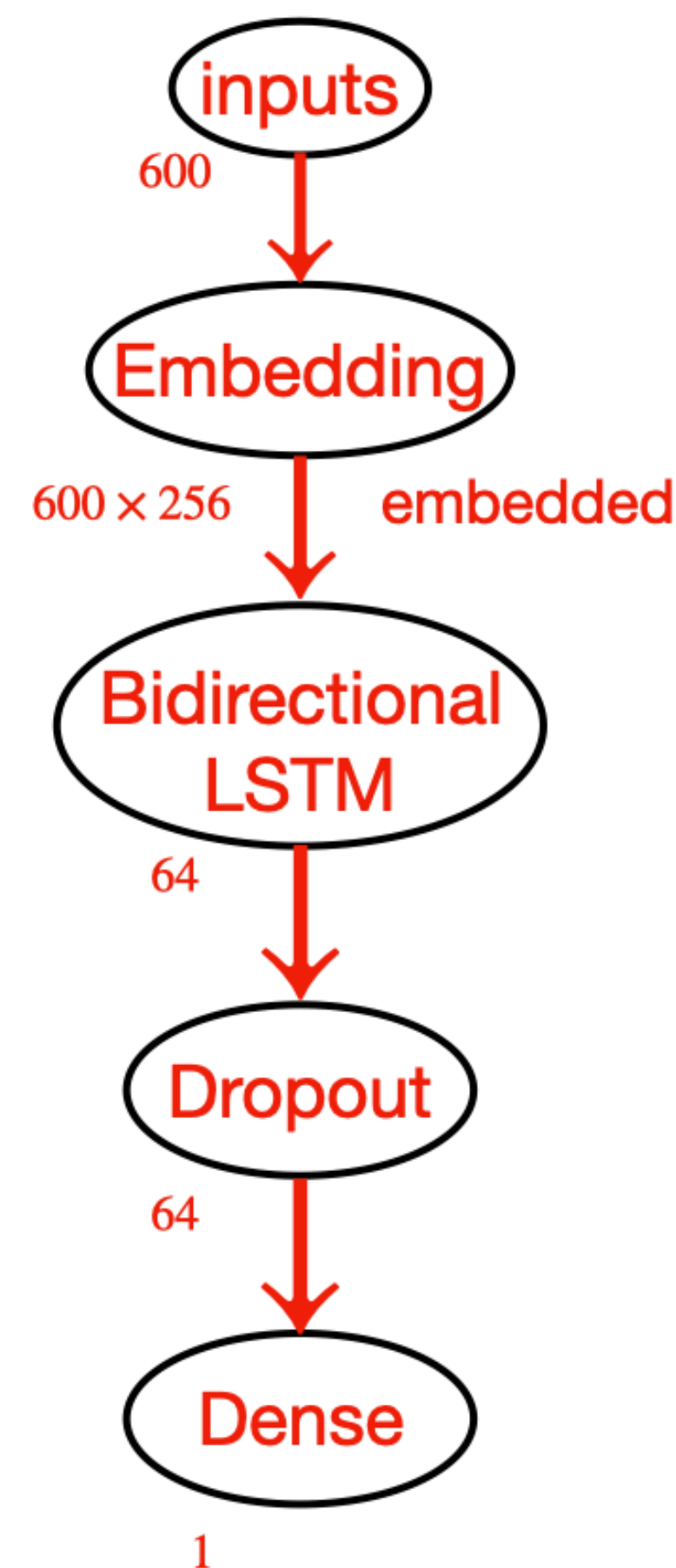
Masking: tell RNN to skip those 0s.

Let's try retraining our model with masking enabled.

Listing 11.17 Using an Embedding layer with masking enabled

```
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = layers.Embedding(
    input_dim=max_tokens, output_dim=256, mask_zero=True)(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()
```

Test accuracy = 88%



Quiz question:

1. What is the benefit of masking?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with the Embedding layer (with masking enabled), Bi-directional LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-directional LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer encoder

1.7 Try 6: Transformer encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation



Popular Word Embeddings

- 1) **Word2Vec**: <https://code.google.com/archive/p/word2vec>
- 2) **GloVe**: <https://nlp.stanford.edu/projects/glove>

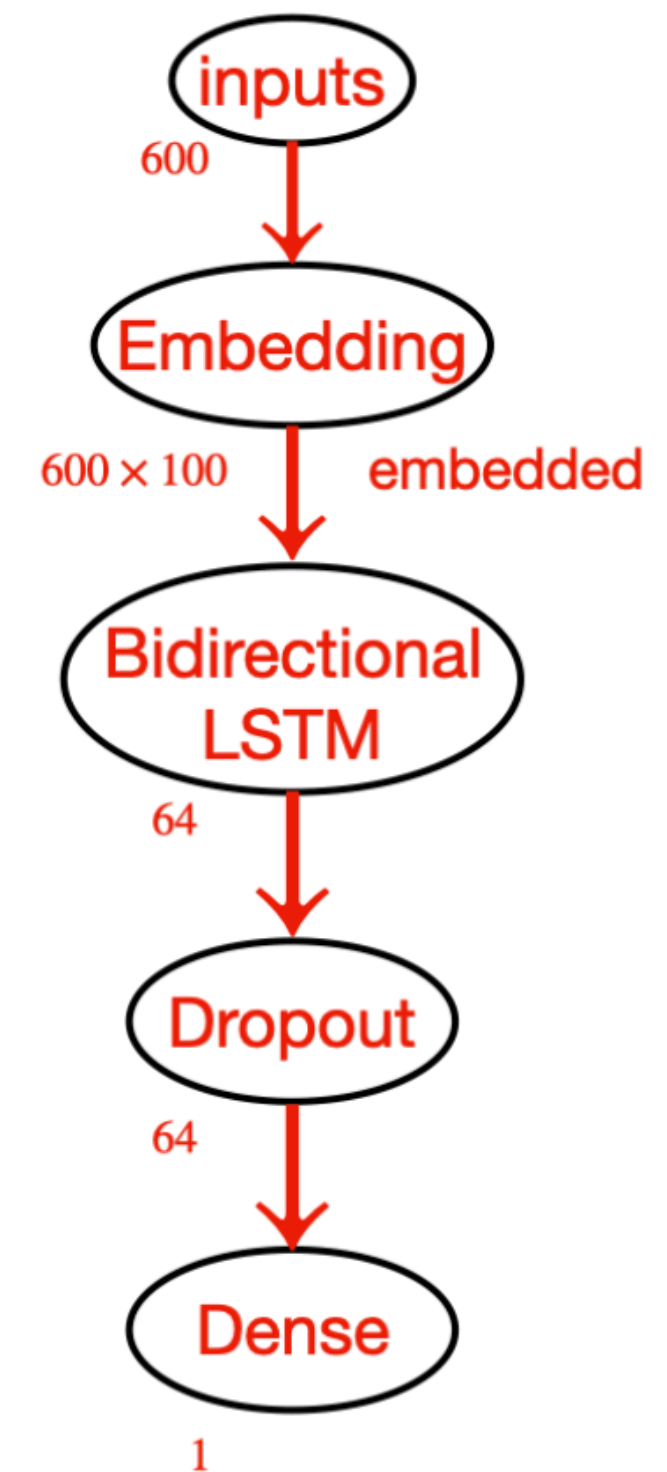
Load **GloVe** into the **Embedding** layer

```
embedding_layer = layers.Embedding(  
    max_tokens,  
    embedding_dim,  
    embeddings_initializer=keras.initializers.Constant(embedding_matrix),  
    trainable=False,  
    mask_zero=True,  
)
```

Listing 11.20 Model that uses a pretrained Embedding layer

```
inputs = keras.Input(shape=(None,), dtype="int64")
embedded = embedding_layer(inputs)
x = layers.Bidirectional(layers.LSTM(32))(embedded)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("glove_embeddings_sequence_model.keras",
                                    save_best_only=True)
]
model.fit(int_train_ds, validation_data=int_val_ds, epochs=10,
        callbacks=callbacks)
model = keras.models.load_model("glove_embeddings_sequence_model.keras")
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```



Performance: no better than the previous approach (but still worth the try)

Quiz question:

1. What is the benefit of pre-trained embedding?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with the Embedding layer (with masking enabled), Bi-directional LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-directional LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer encoder

1.7 Try 6: Transformer encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation



Attention and Transformer

Transformer: sequence-to-sequence model
with “self-attention”

Comparison between different layers

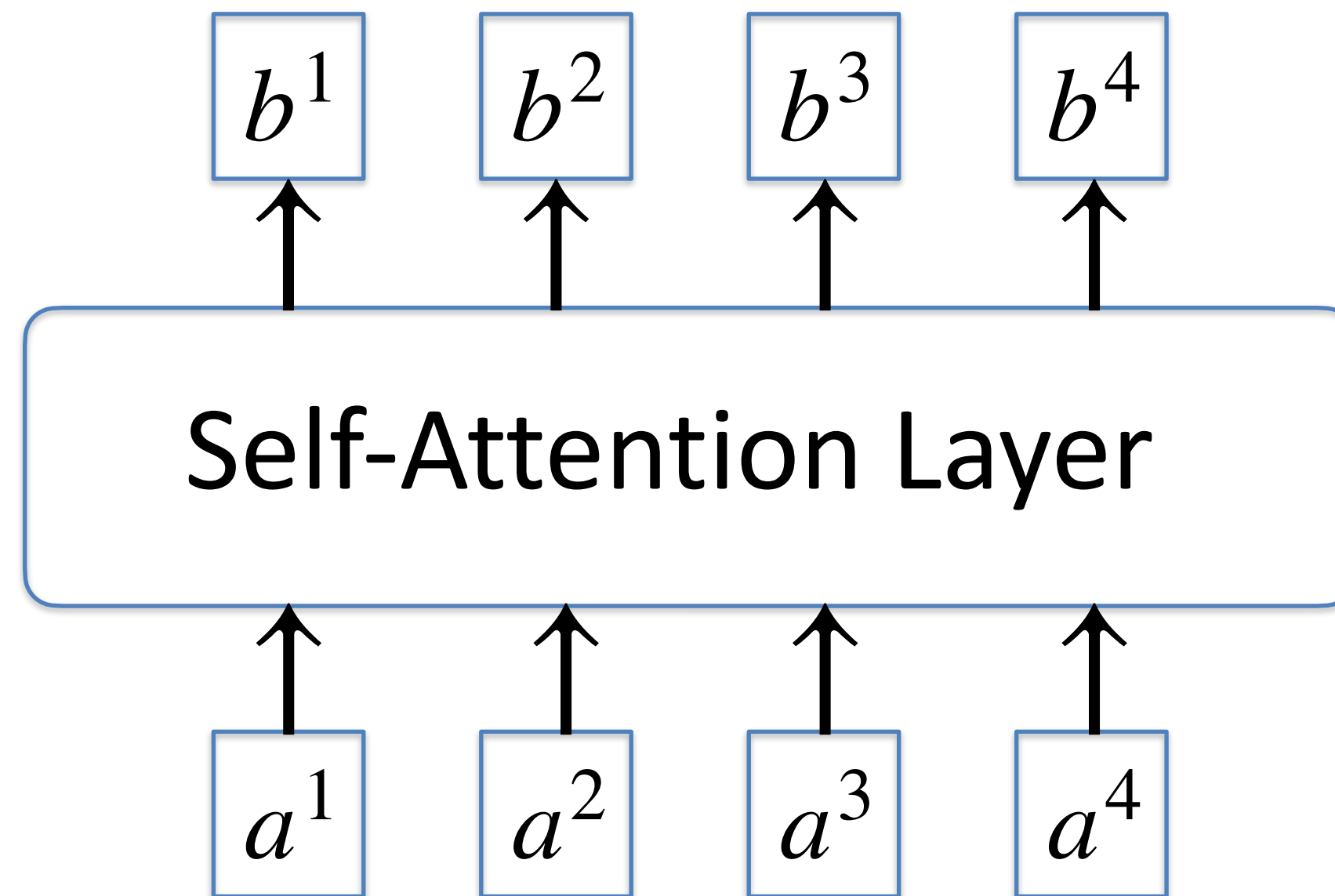
RNN layer: captures global information, but *hard to use parallel computing, slow*

CNN layer: fewer parameters to train (filter weights are shared), supports parallel computing, fast, but *does not capture global information*

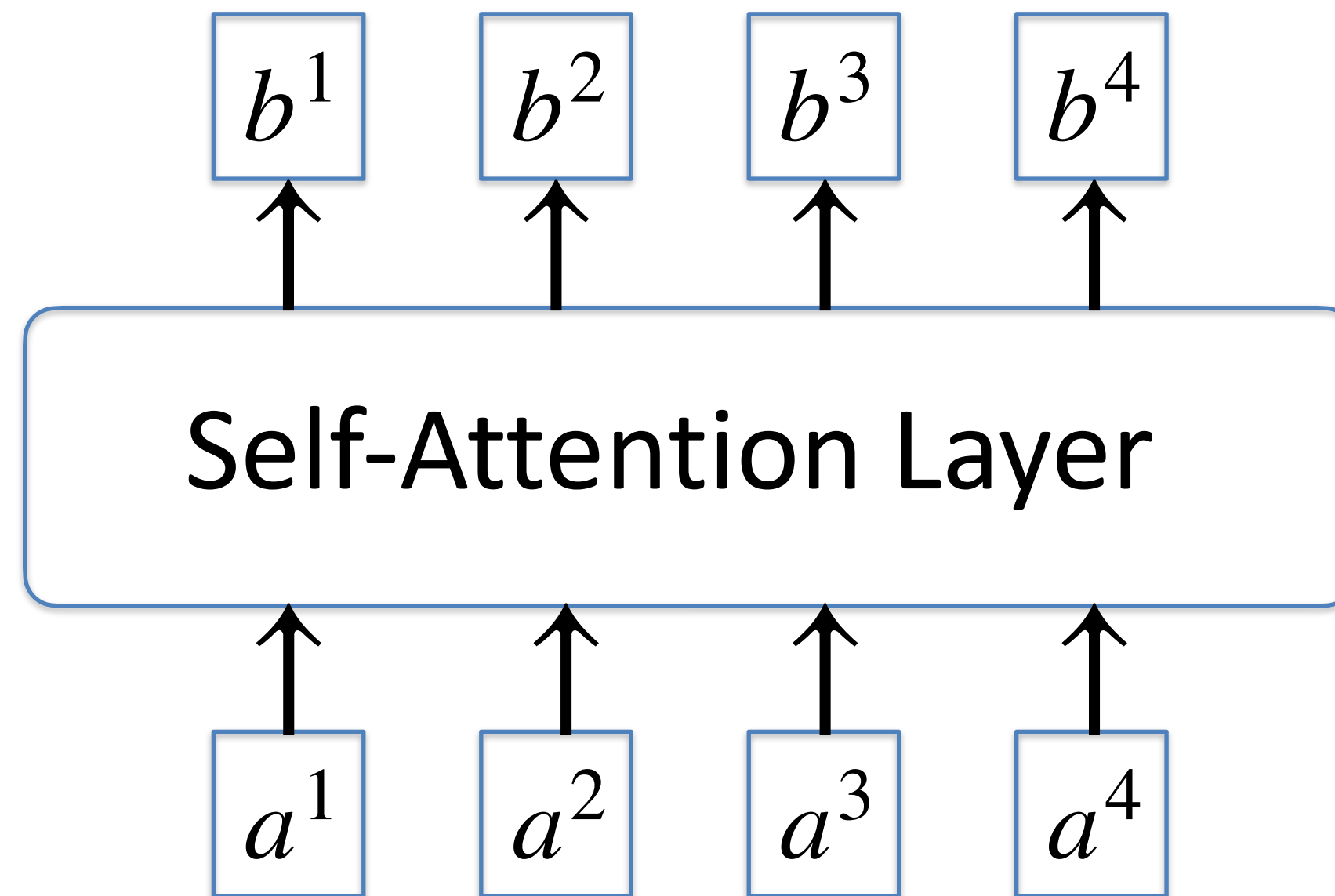
Dense layer: captures global information, but *hard to use parallel computing, slow, too many parameters to train*

Attention layer: captures global information, few parameters to train (weights are shared), supports parallel computing, fast

Self-Attention Layer for Sequence-to-Sequence Learning



Self-Attention Layer for Sequence-to-Sequence Learning



Each output symbol is obtained using all the input symbols.

The output symbols can be computed in parallel.

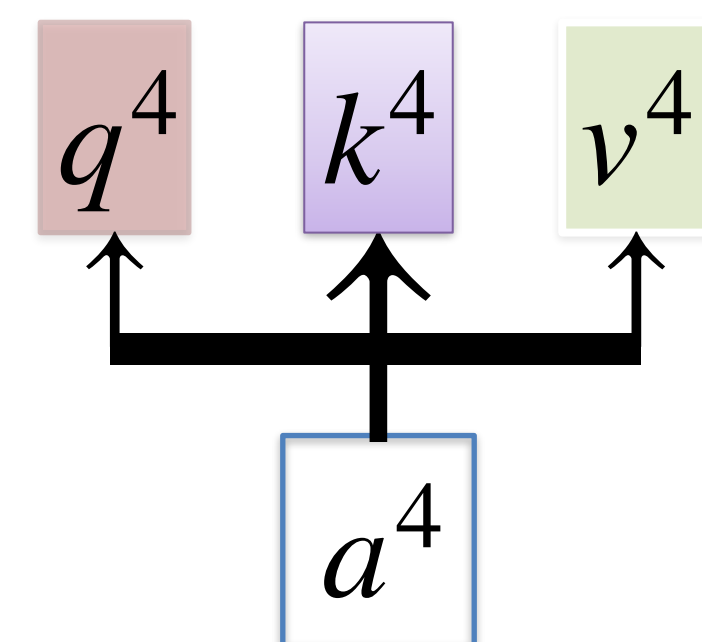
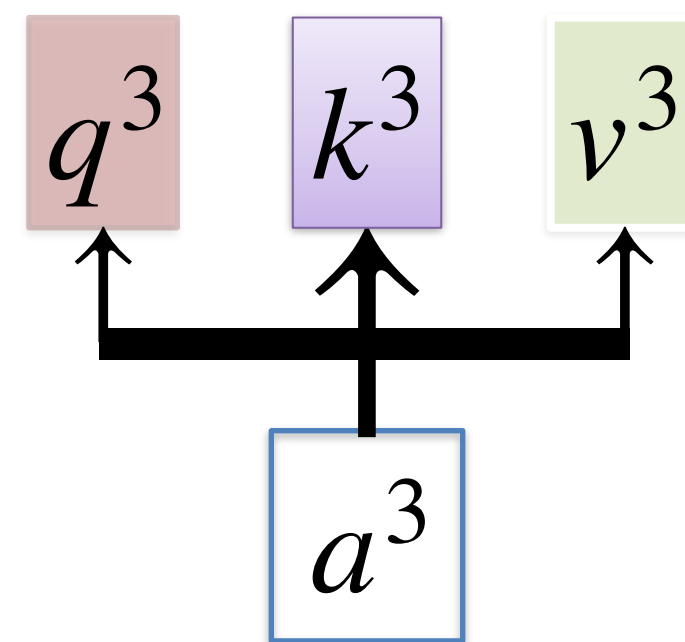
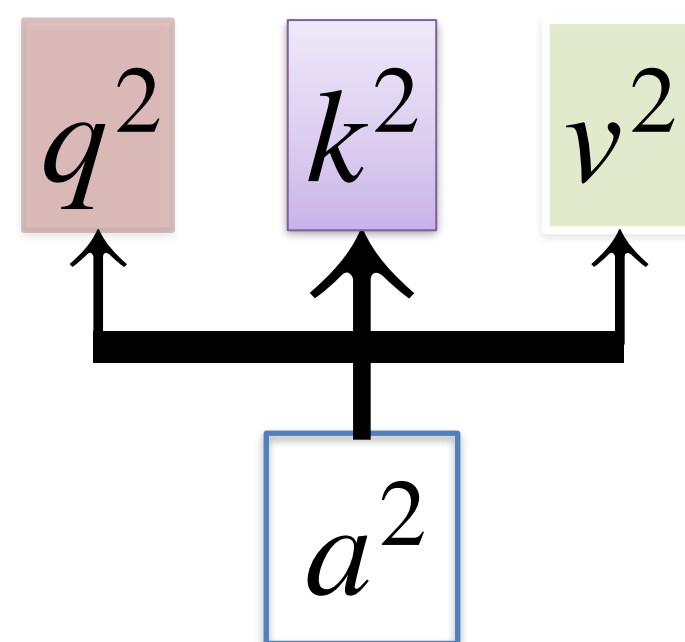
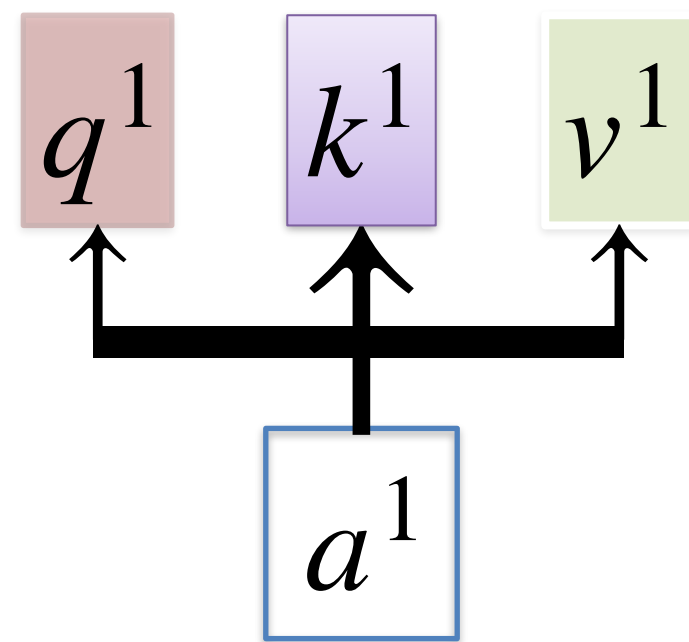
q^i : query vector

k^i : key vector

v^i : value vector

a^i : input vector

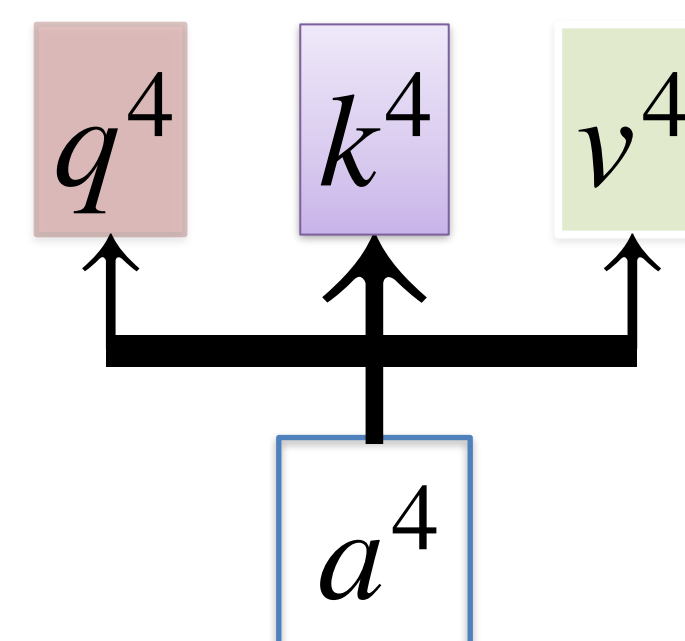
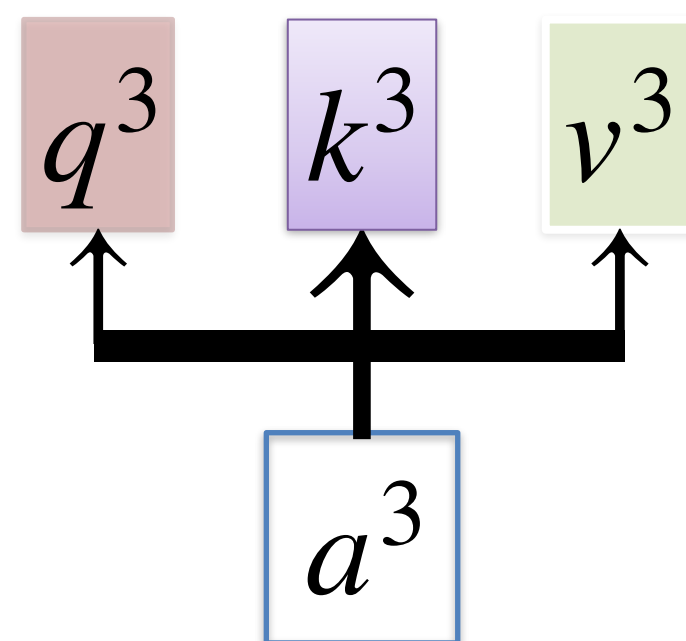
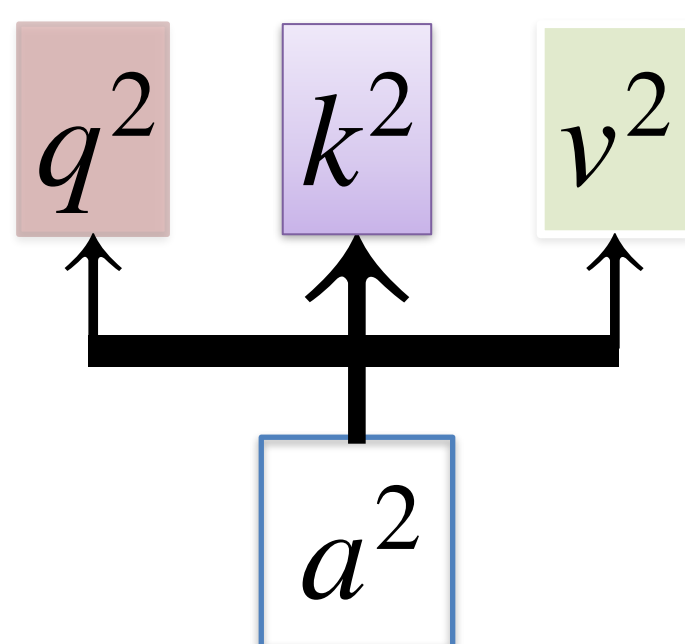
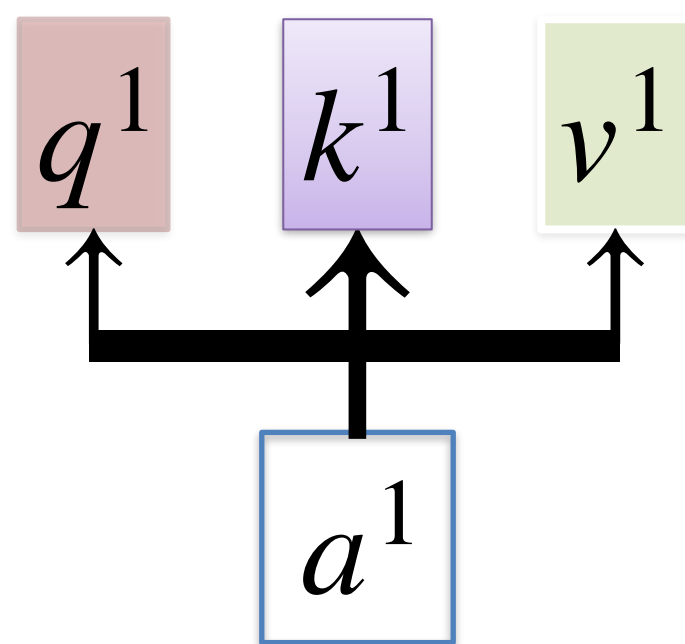
3 linear transformations to be learned,
which are shared for different i



$$q^i = W^q a^i$$

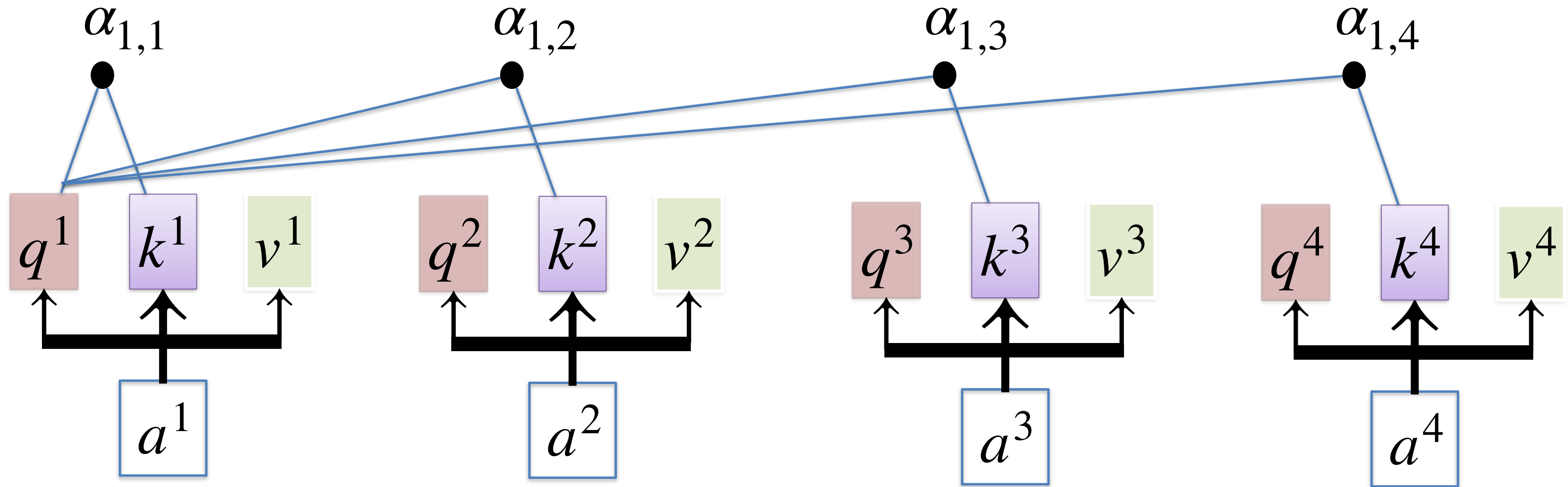
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

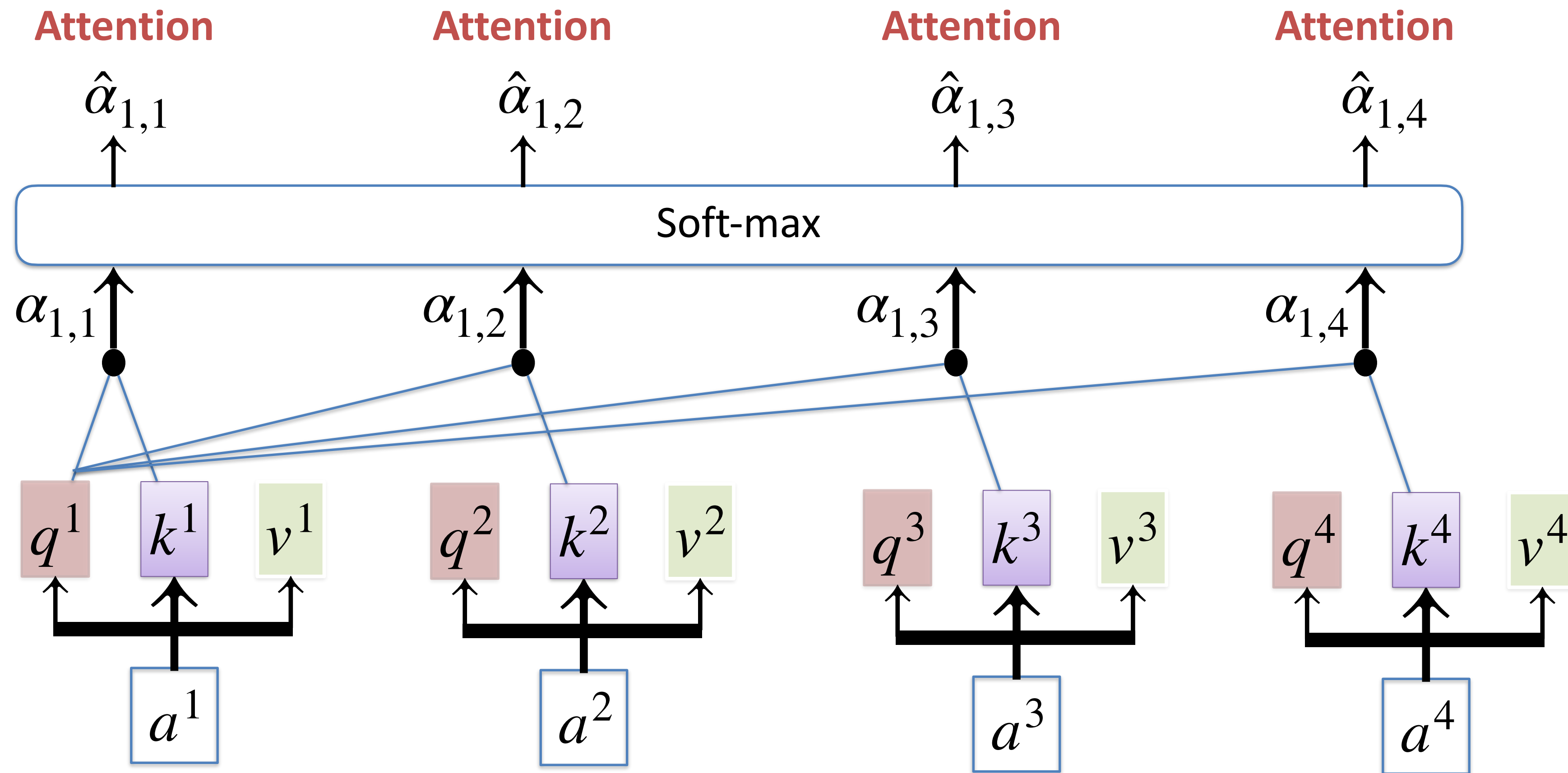


Self-attention

Scaled dot-product attention: $\alpha_{1,i} = \frac{q^1 \cdot k^i}{\sqrt{d}}$, where d is the dimension of q and k



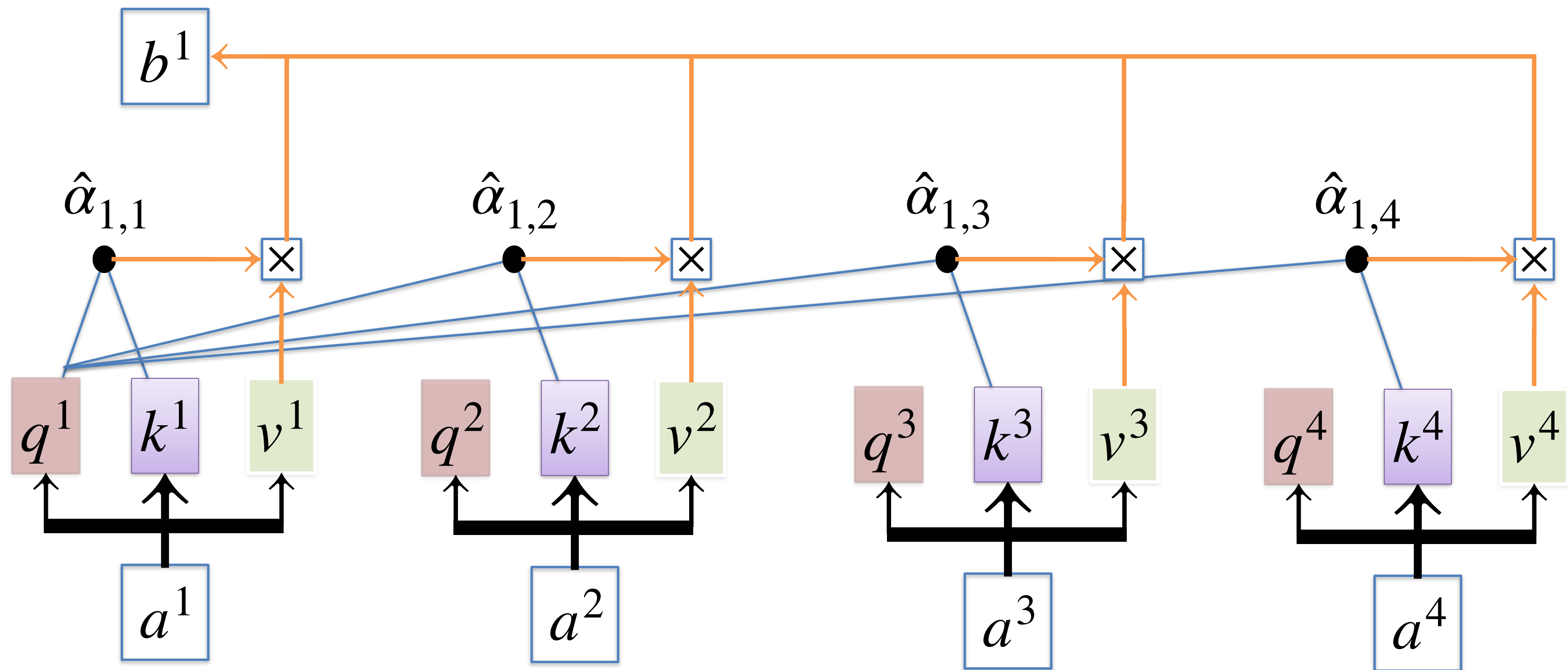
$$\hat{\alpha}_{1,i} = \frac{e^{\alpha_{1,i}}}{\sum_j e^{\alpha_{1,j}}}$$

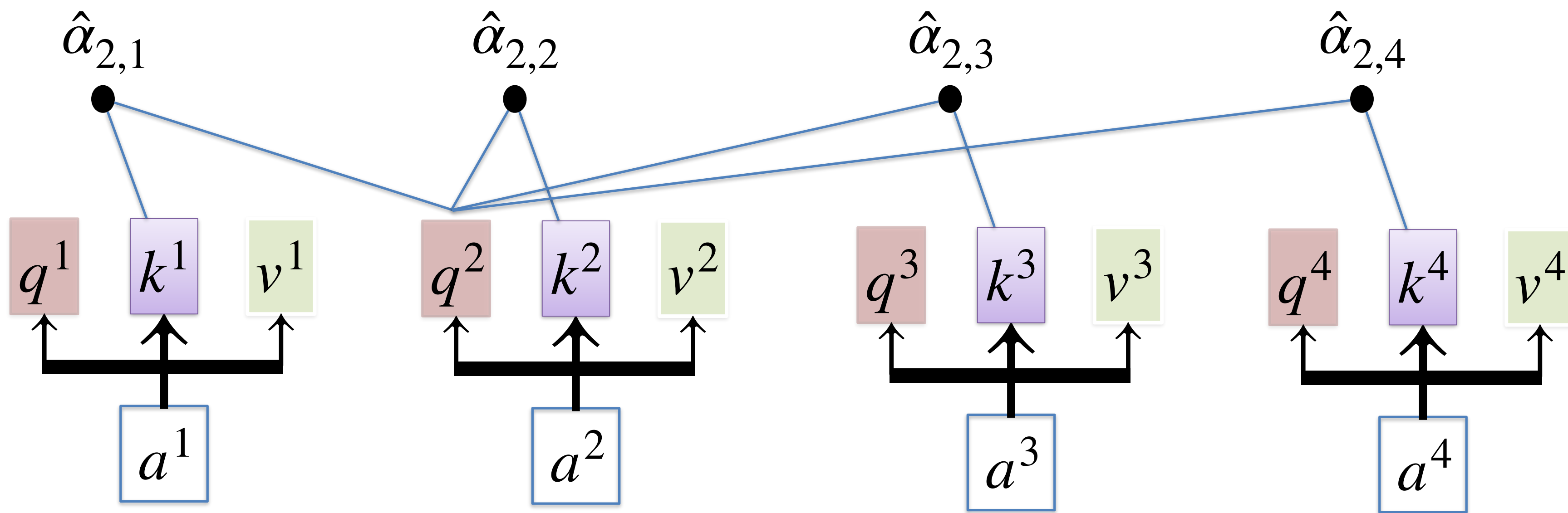


Example: "The train has left the station."

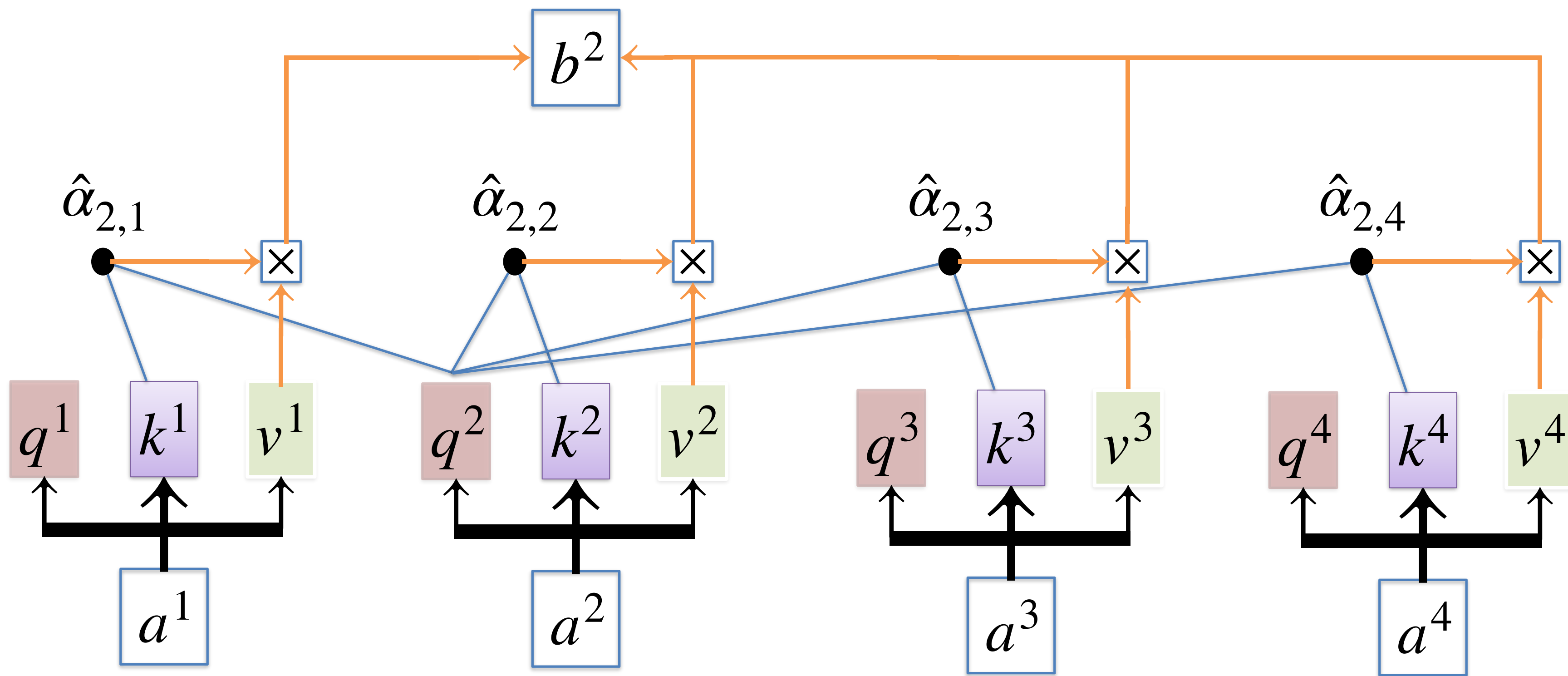
$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$

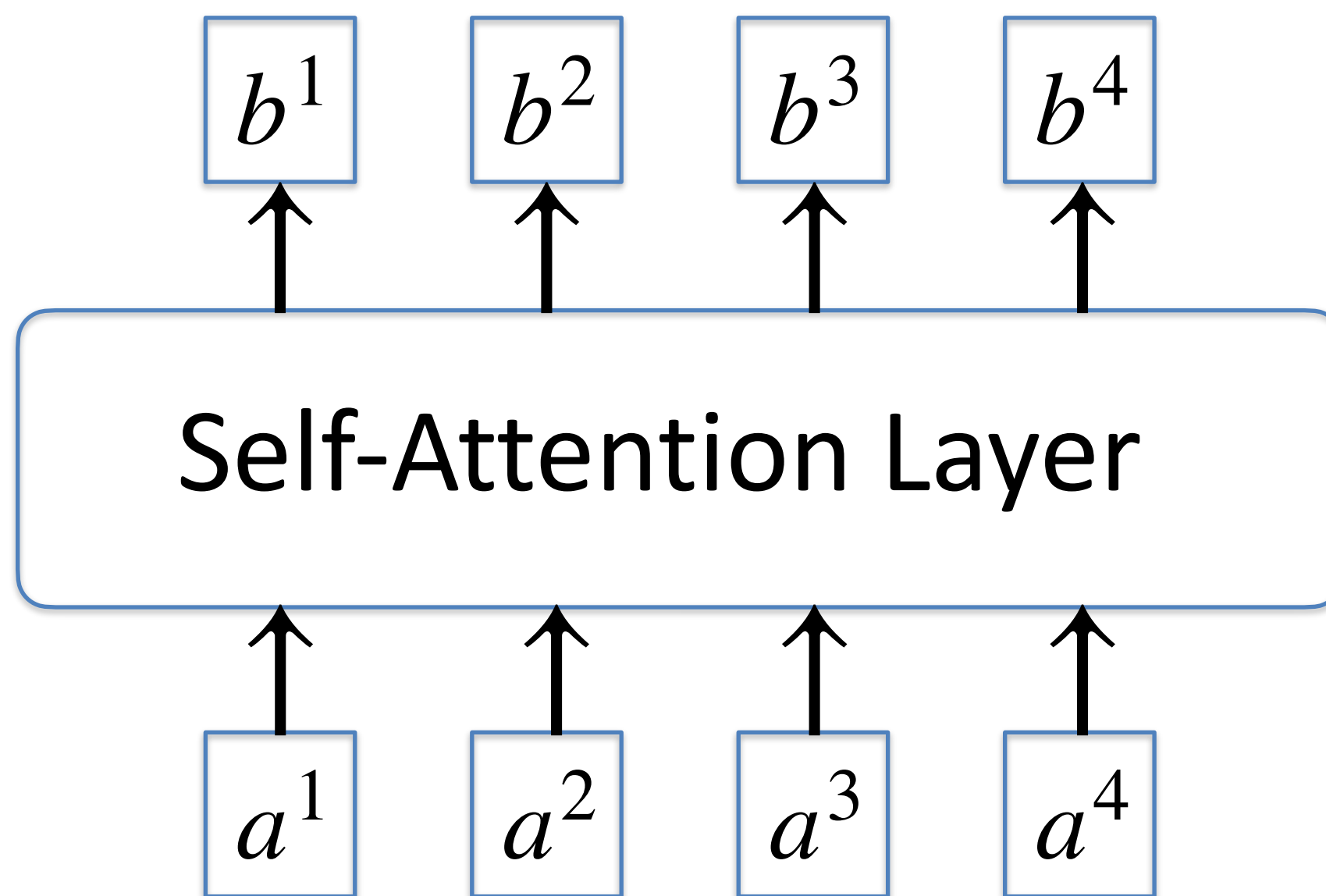
b^1 considers the whole input sequence.

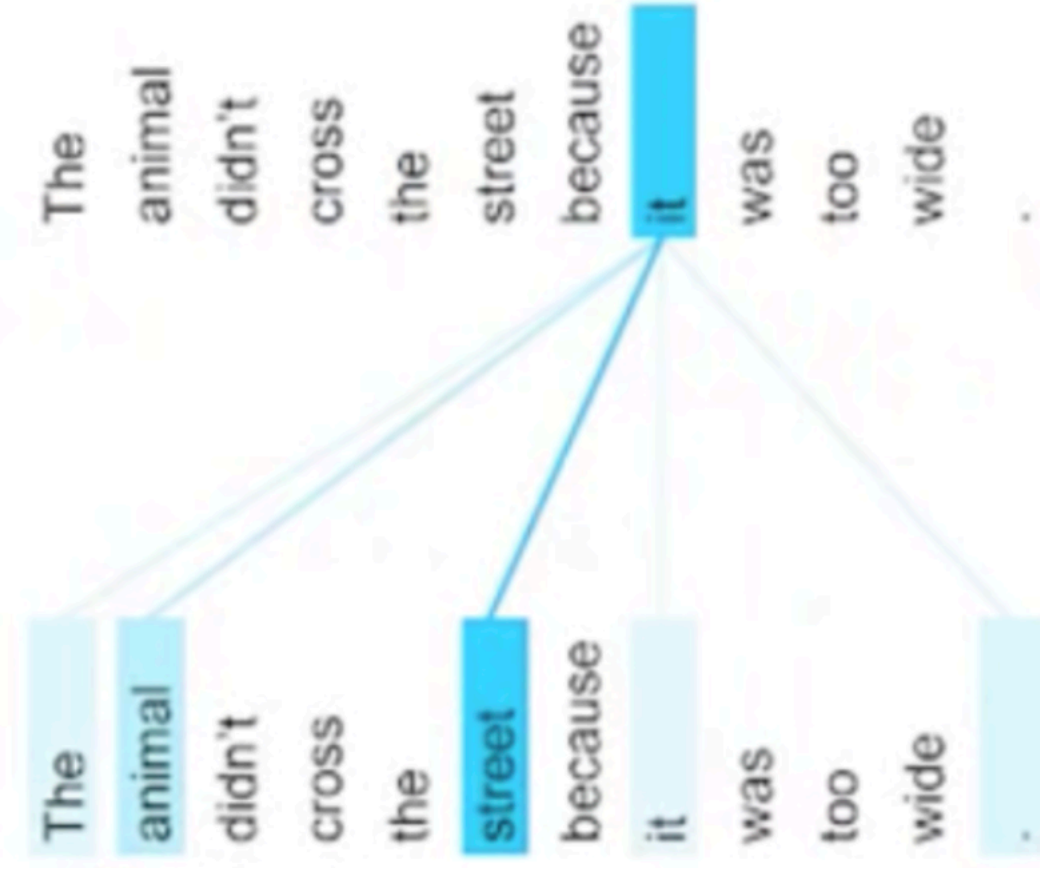
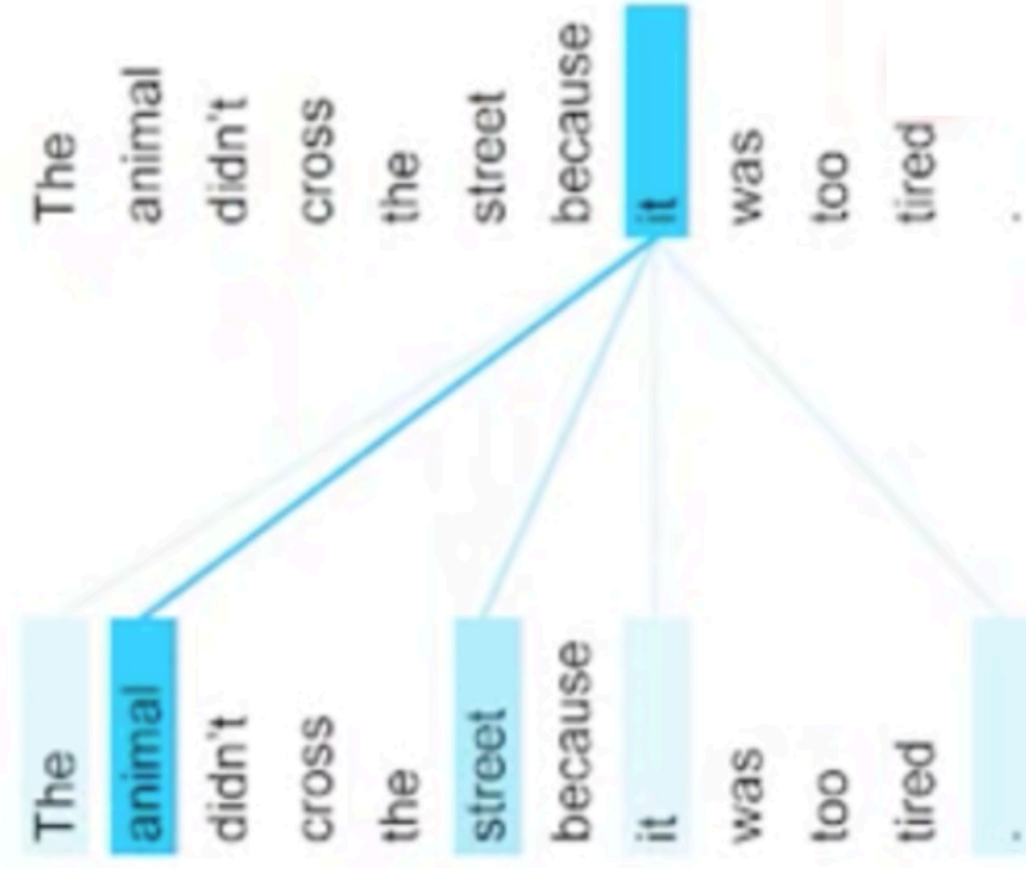




$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



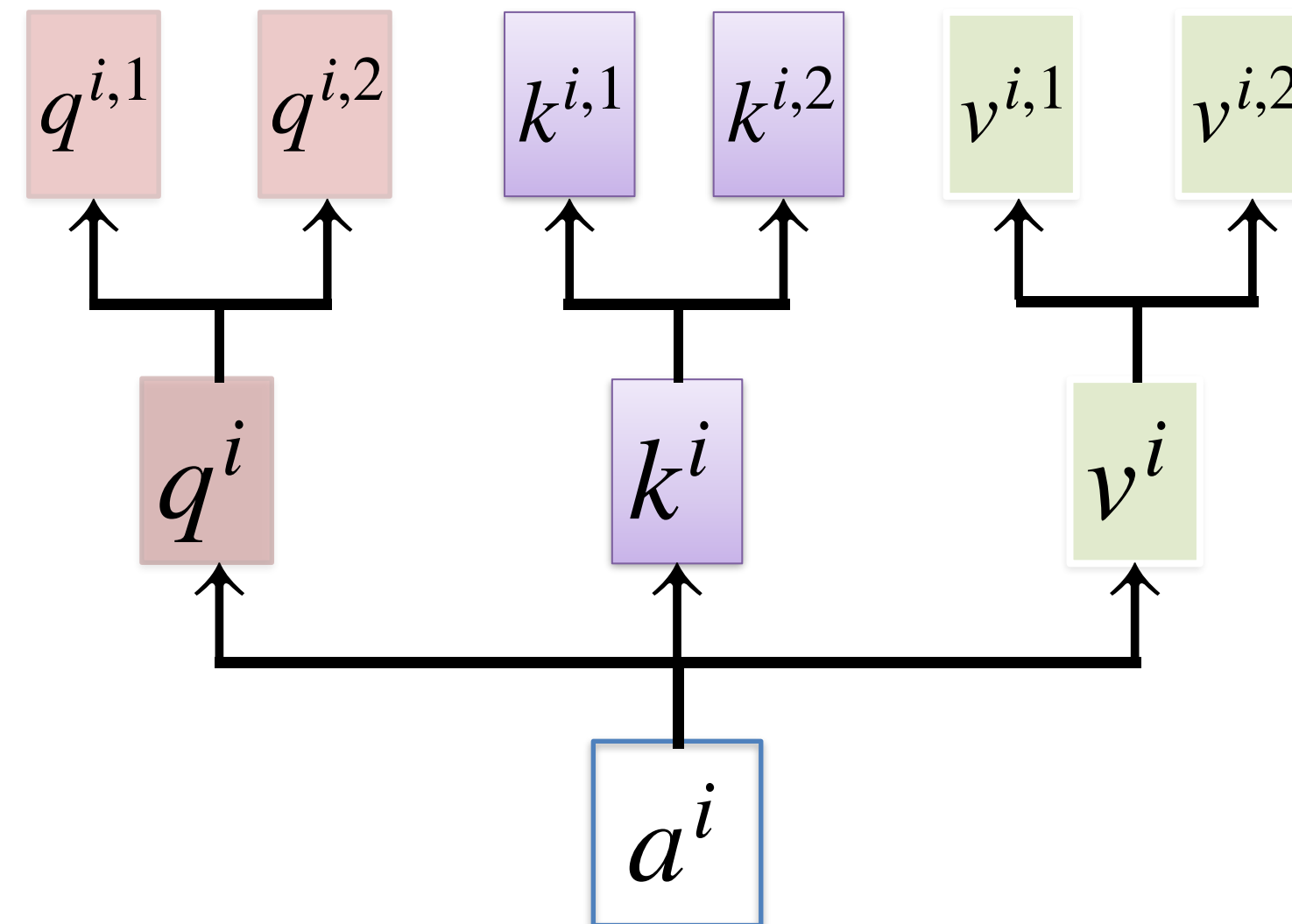




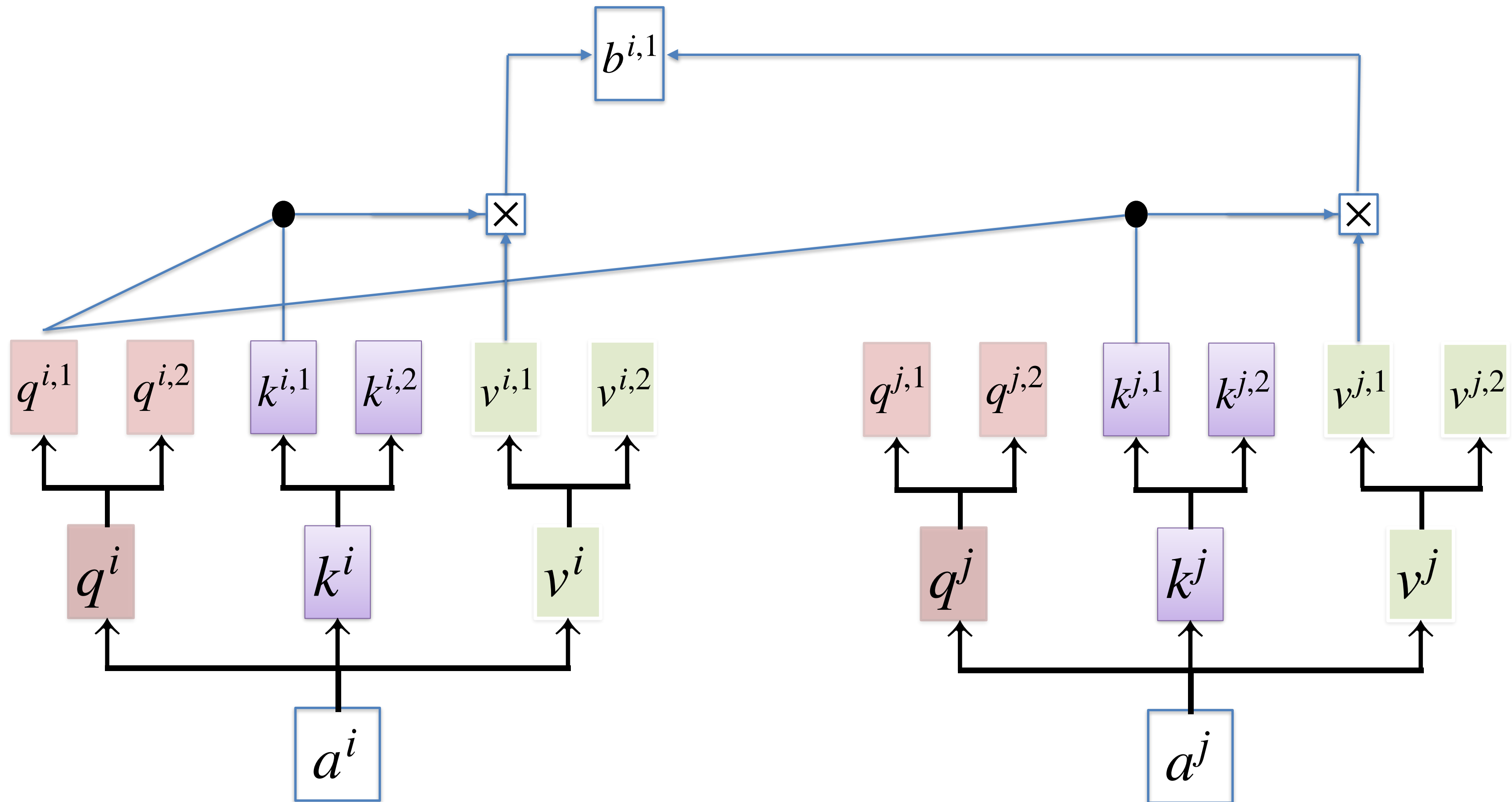
Multi-head Self-attention

Example: 2 heads

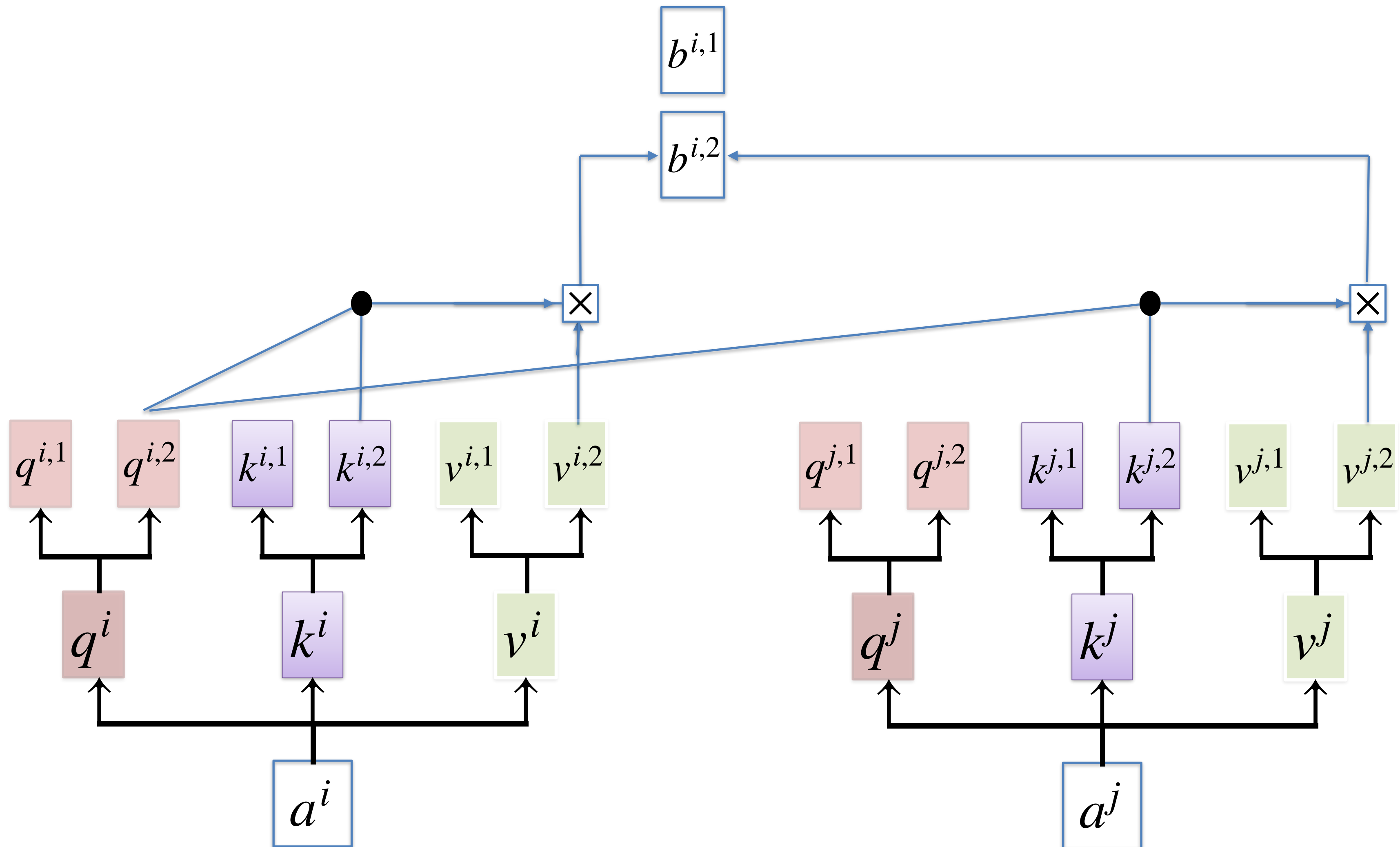
$$\begin{aligned} q_{i,1} &= W^{q,1} q^i & k_{i,1} &= W^{k,1} k^i & v_{i,1} &= W^{v,1} v^i \\ q_{i,2} &= W^{q,2} q^i & k_{i,2} &= W^{k,2} k^i & v_{i,2} &= W^{v,2} v^i \end{aligned}$$



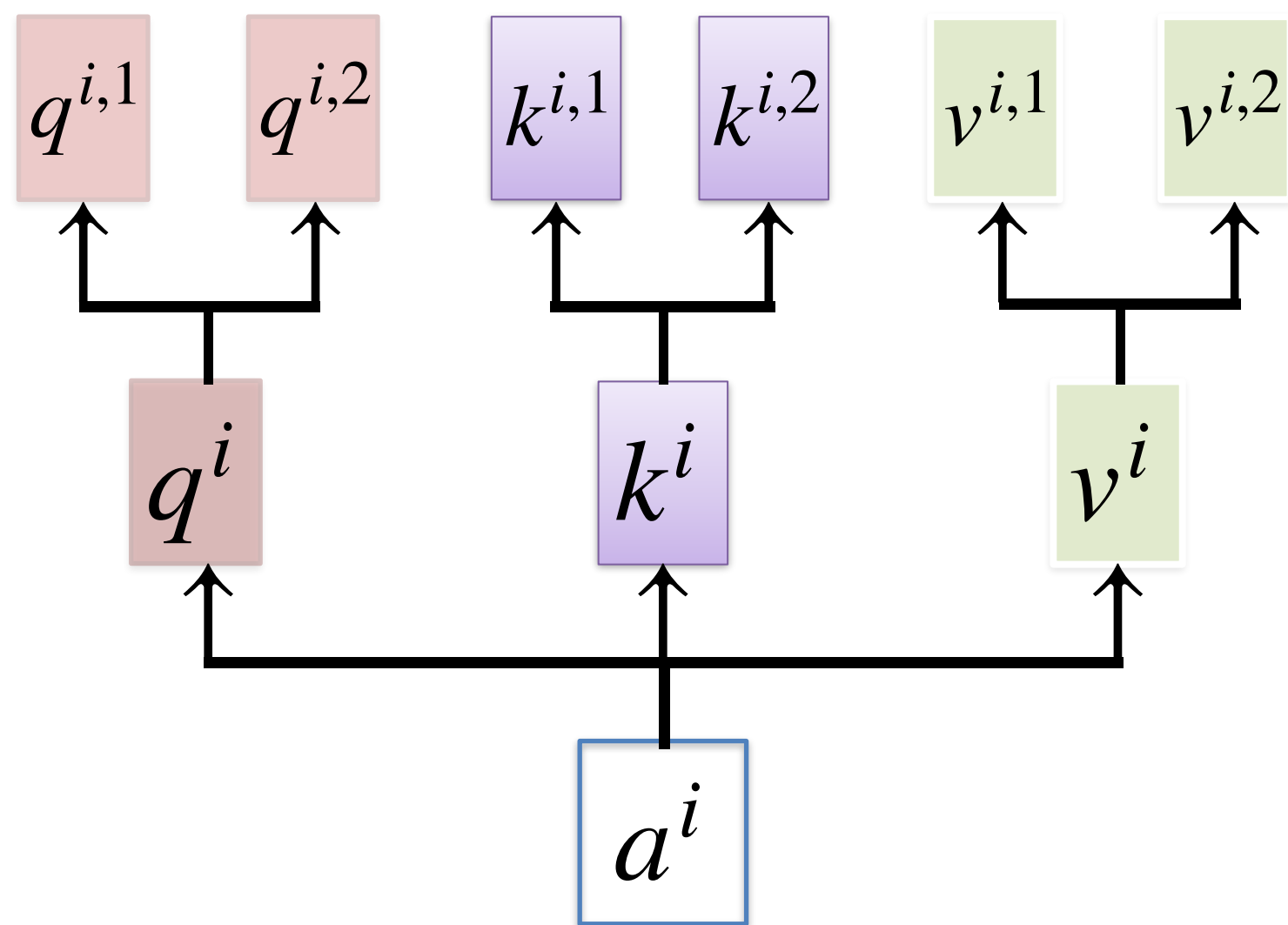
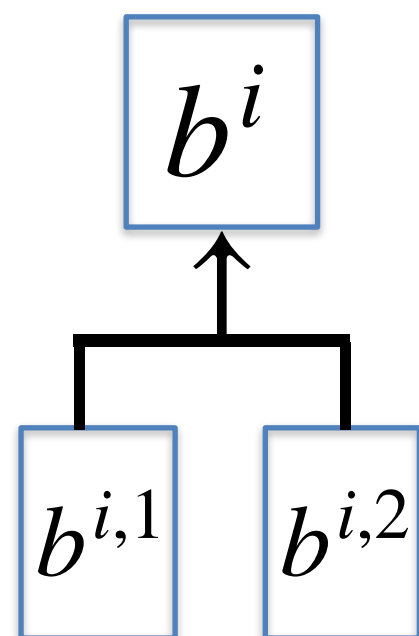
query 1 only interacts with key 1 and value 1



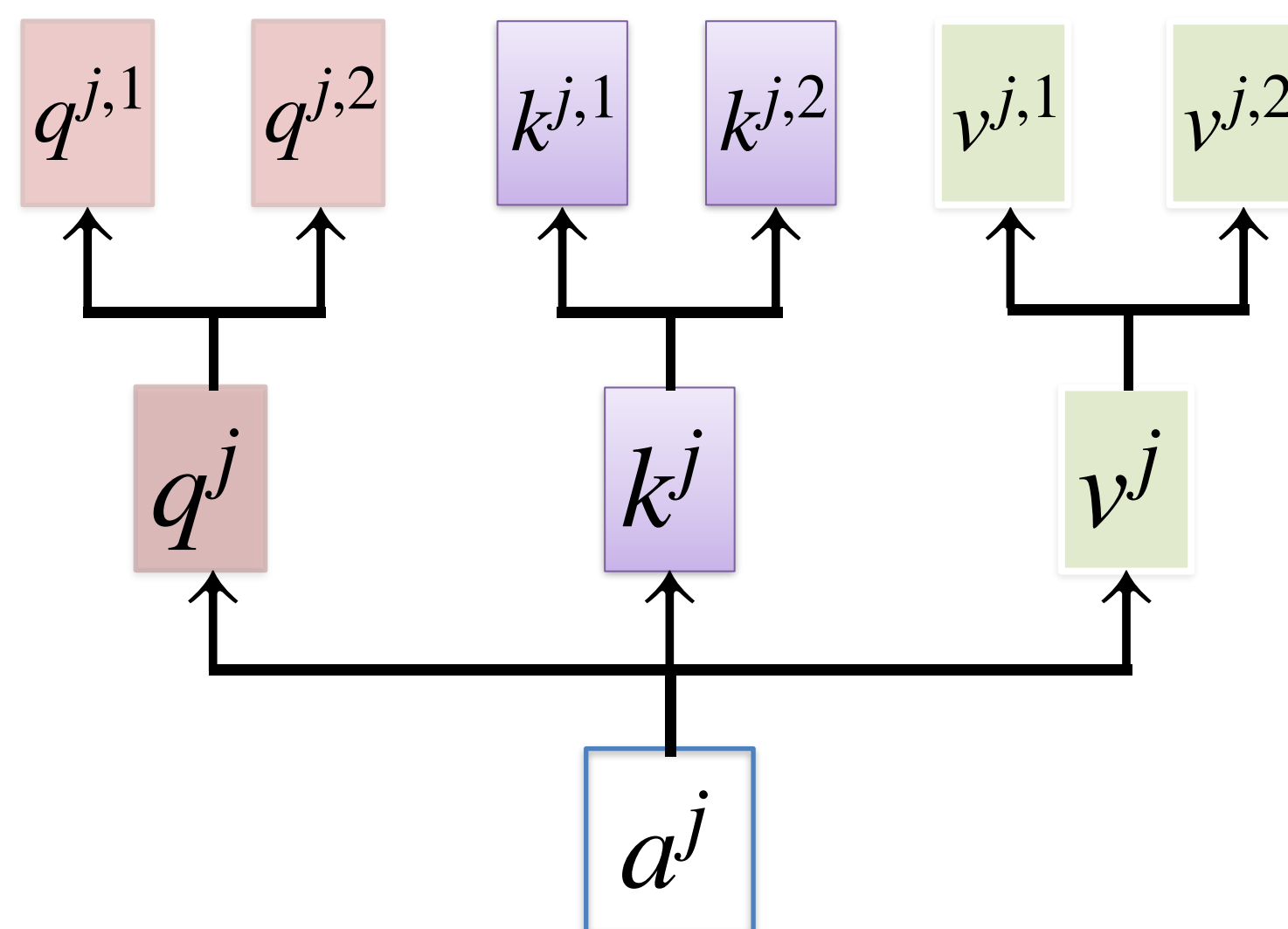
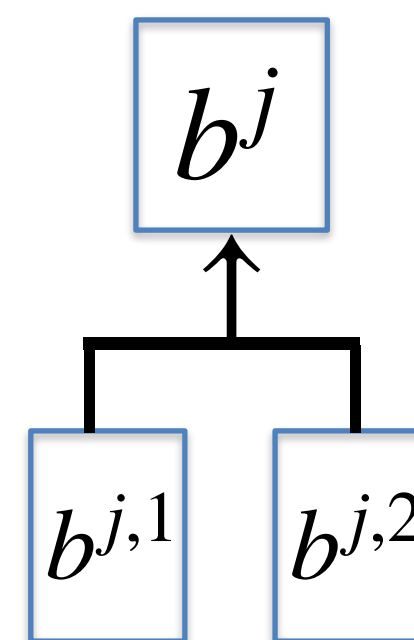
query 2 only interacts with key 2 and value 2



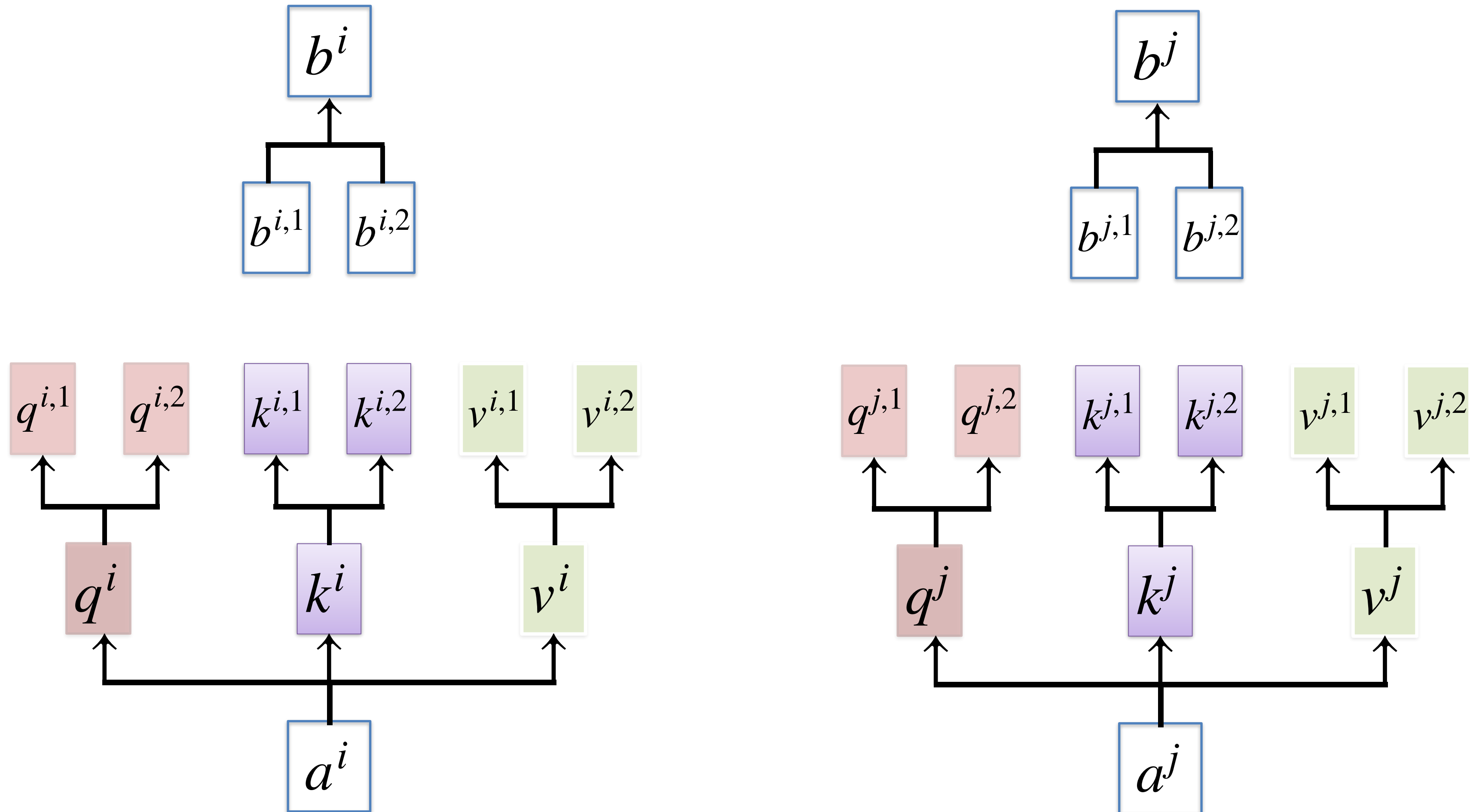
$$b^i = W^o \begin{bmatrix} b^{i,1} \\ b^{i,2} \end{bmatrix}$$

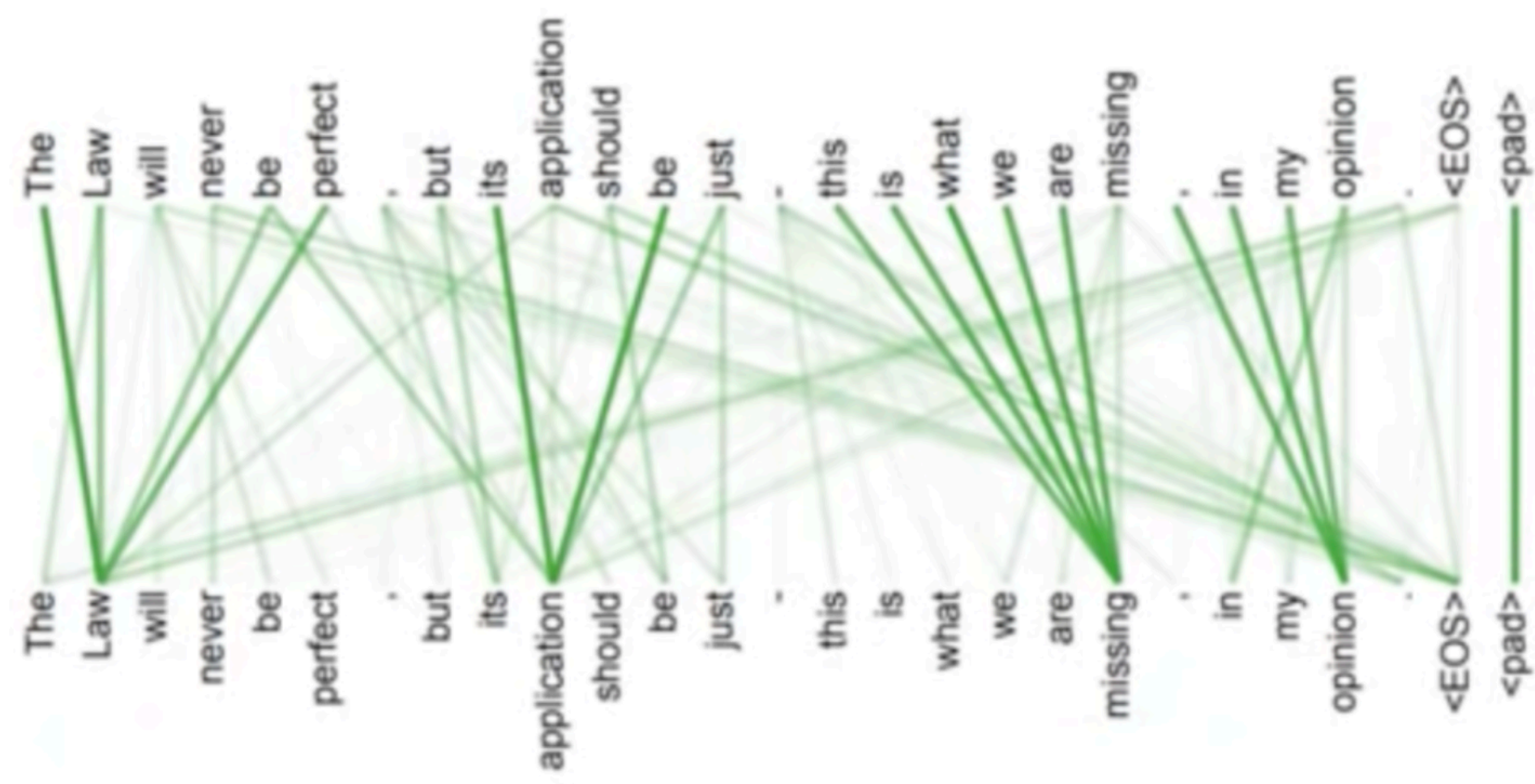
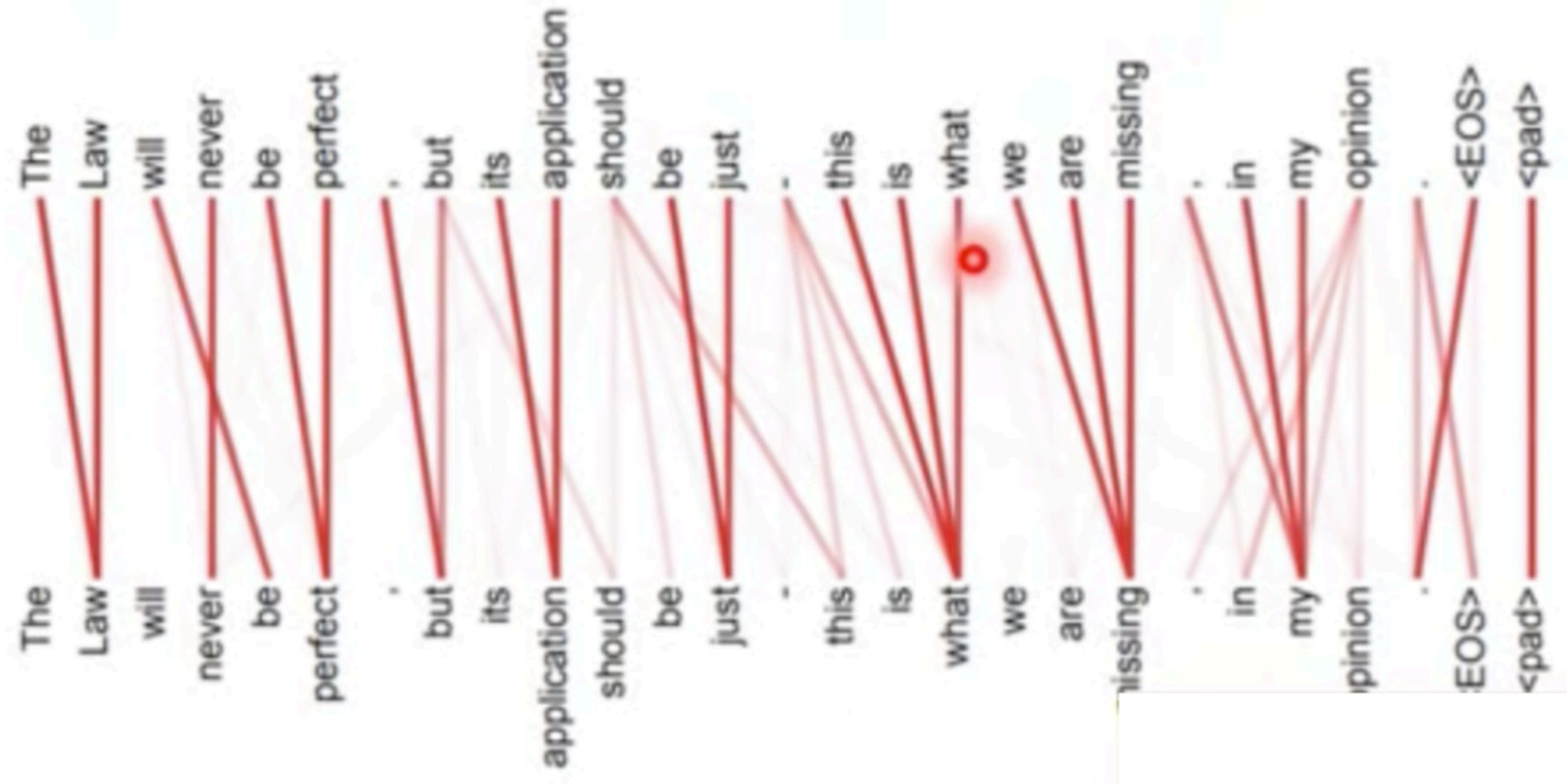


$$b^j = W^o \begin{bmatrix} b^{j,1} \\ b^{j,2} \end{bmatrix}$$

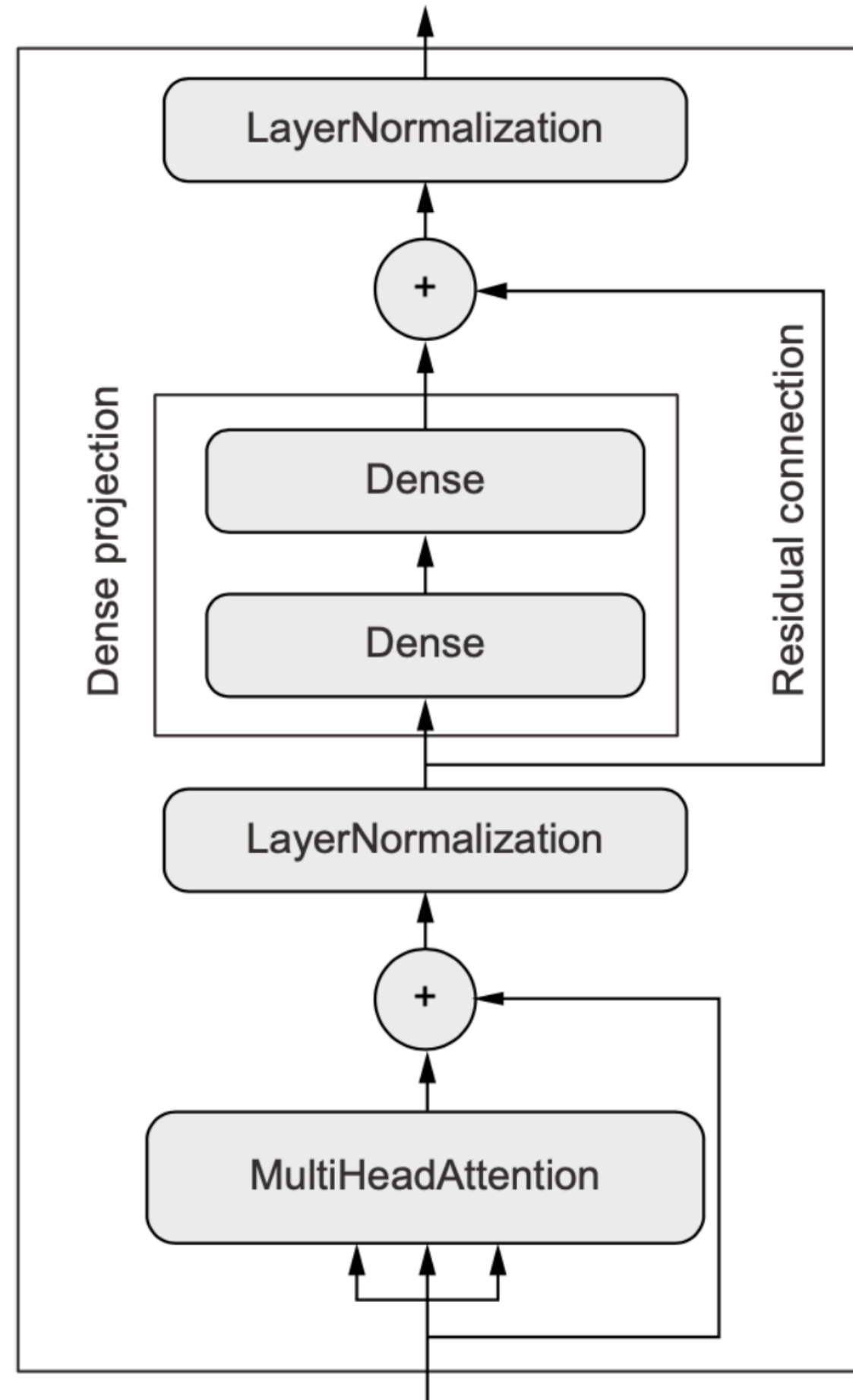


Benefit of multi-heads: different heads use different kinds of attention
(e.g., local attention, global attention, etc.)





The Transformer Encoder



Quiz questions:

1. What is self-attention?
2. What is the architecture of the Transformer encoder?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with the Embedding layer (with masking enabled), Bi-directional LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-directional LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer Encoder

1.7 Try 6: Transformer encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation



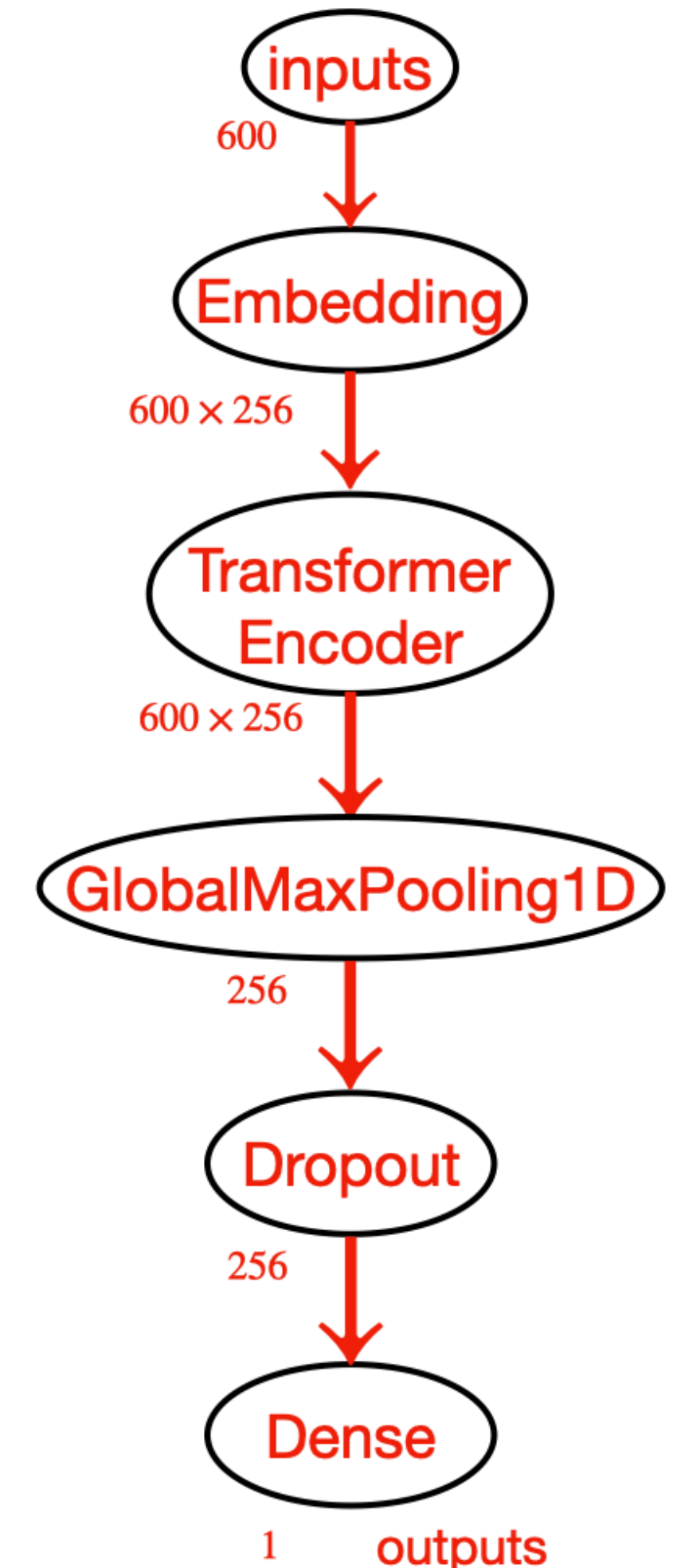
Now that we've implemented our TransformerEncoder, we can use it to assemble a text-classification model similar to the GRU-based one you've seen previously.

Listing 11.22 Using the Transformer encoder for text classification

```
vocab_size = 20000
embed_dim = 256
num_heads = 2
dense_dim = 32
```

```
inputs = keras.Input(shape=(None,), dtype="int64")
x = layers.Embedding(vocab_size, embed_dim)(inputs)
x = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
x = layers.GlobalMaxPooling1D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()
```

Since TransformerEncoder returns full sequences, we need to reduce each sequence to a single vector for classification via a global pooling layer.



Test accuracy = 87.5%

Quiz question:

1. What are the pros and cons of the above approach using the transformer encoder?

Roadmap of this lecture:

1. Text classification for IMDB

1.1 Try 1: Sequence approach, one-hot encoding, Bi-directional LSTM

1.2 Try 2: Sequence approach, learn word embedding with the Embedding layer, Bi-directional LSTM

1.3 Try 3: Sequence approach, learn word embedding with the Embedding layer (with masking enabled), Bi-directional LSTM



Roadmap of this lecture (continued):

1.4 Try 4: Sequence approach, pre-trained word embedding, Bi-directional LSTM

1.5 Attention and Transformer

1.6 Try 5: Transformer Encoder

1.7 Try 6: Transformer Encoder with positional embedding

2. Sequence-to-Sequence Learning for Machine Translation



Using **Positional Embedding** to re-inject order information

Embed the position of each word
(e.g., by the cosine of position, or learn the positional embedding)

Add the positional-embedding vector to the word-embedding vector.

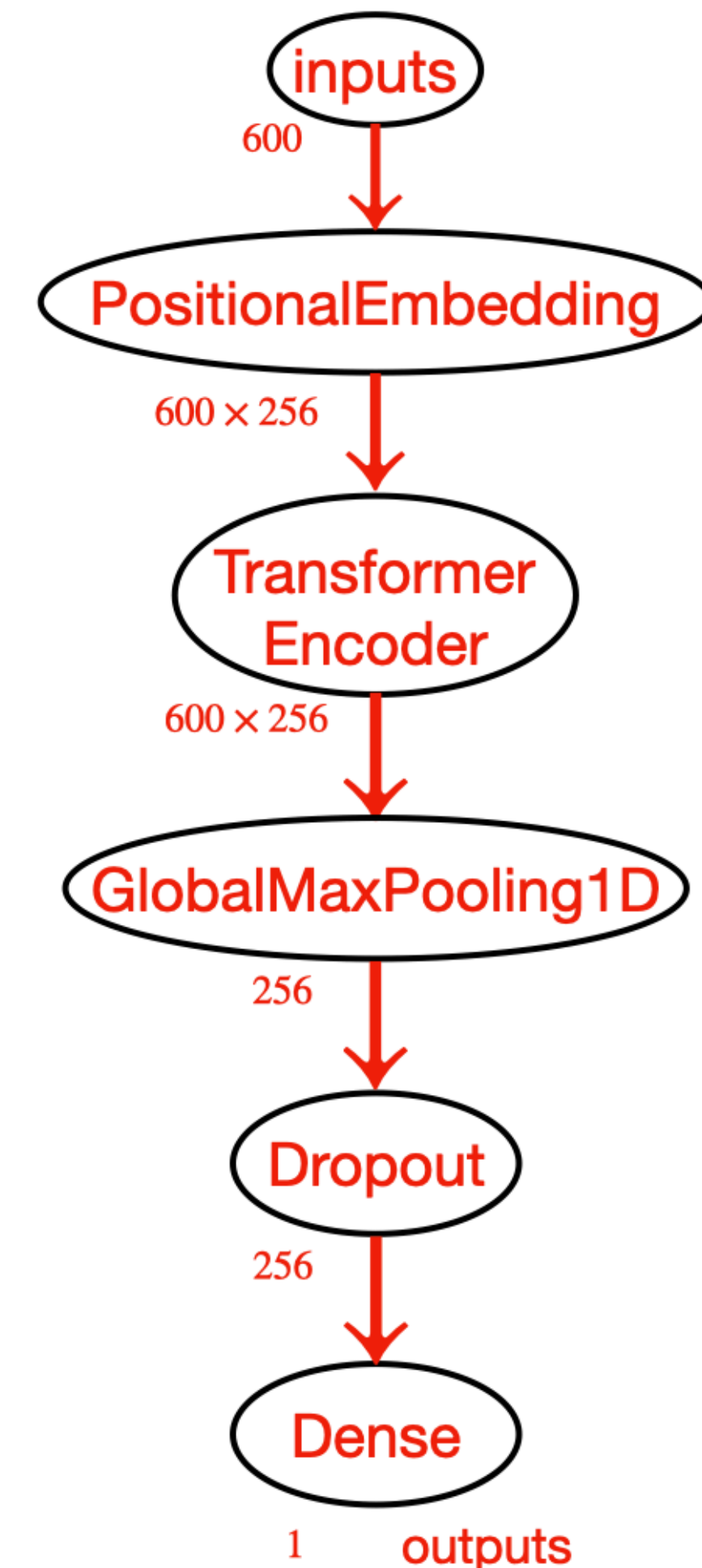
Listing 11.25 Combining the Transformer encoder with positional embedding

```
vocab_size = 20000
sequence_length = 600
embed_dim = 256
num_heads = 2
dense_dim = 32

inputs = keras.Input(shape=(None,), dtype="int64")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(inputs)
x = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
x = layers.GlobalMaxPooling1D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("full_transformer_encoder.keras",
                                    save_best_only=True)
]
model.fit(int_train_ds, validation_data=int_val_ds, epochs=20,
        callbacks=callbacks)
model = keras.models.load_model(
    "full_transformer_encoder.keras",
    custom_objects={"TransformerEncoder": TransformerEncoder,
                    "PositionalEmbedding": PositionalEmbedding})
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

Look here!



Test accuracy = 88.3%

Quiz question:

- I. What are the pros and cons of using transformer encoder with positional embedding?

Roadmap of this lecture:

1. Text classification for IMDB

2. Sequence-to-Sequence Learning for Machine Translation

2.1 Try 1 for Machine Translation: RNN

2.2 Try 2 for Machine Translation: Transformer



Sequence-to-Sequence Learning

A seq-2-seq model takes a sequence as input, and translates it into a different sequence.

Examples:

- Machine translation
- Text summarization
- Question answering
- Chatbots
- Text generation given prompt

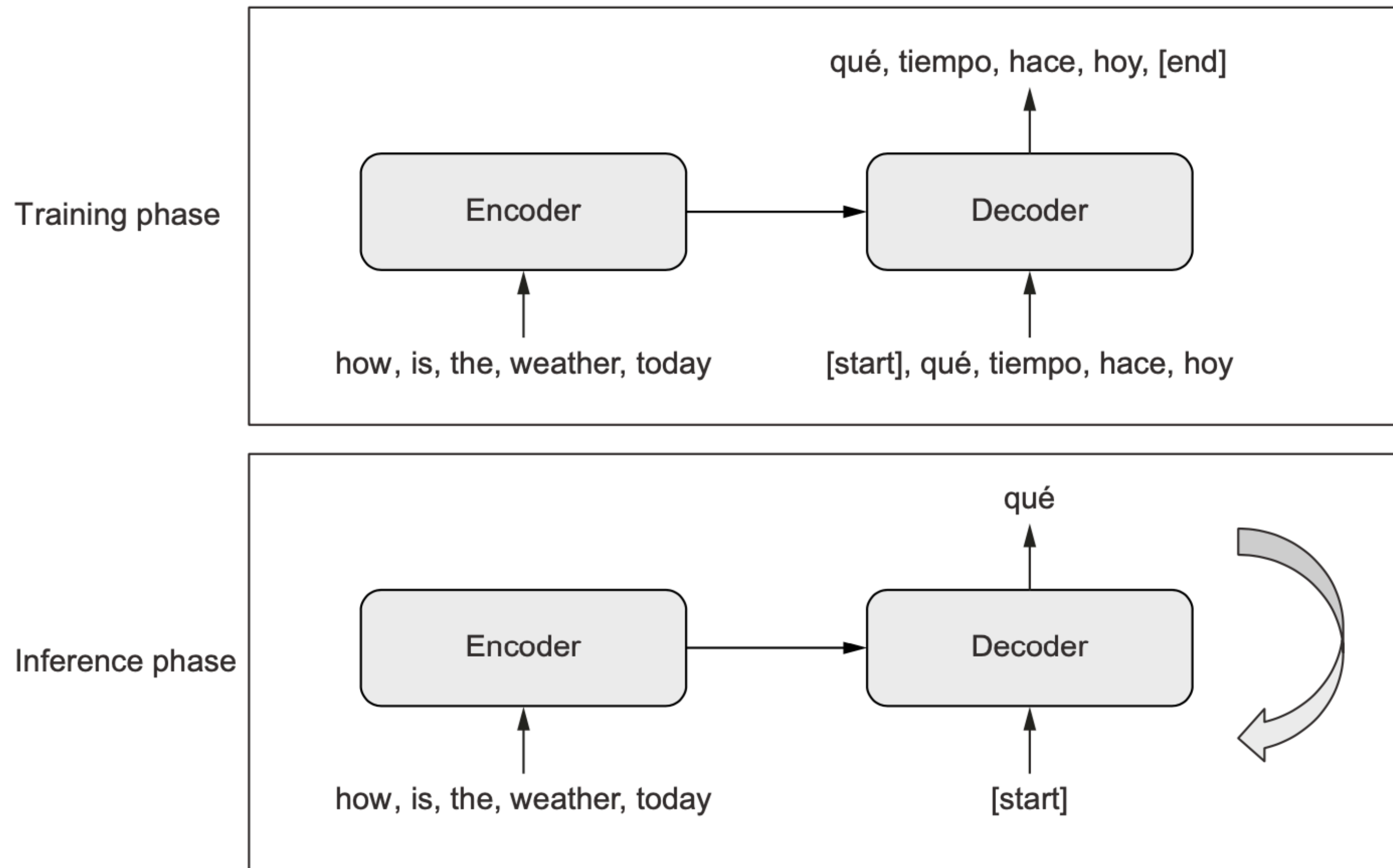


Figure 11.12 Sequence-to-sequence learning: the source sequence is processed by the encoder and is then sent to the decoder. The decoder looks at the target sequence so far and predicts the target sequence offset by one step in the future. During inference, we generate one target token at a time and feed it back into the decoder.

Quiz questions:

1. What is sequence-to-sequence (seq-2-seq) learning?
2. What are some important applications of seq-2-seq learning?

Roadmap of this lecture:

1. Text classification for IMDB

2. Sequence-to-Sequence Learning for Machine Translation

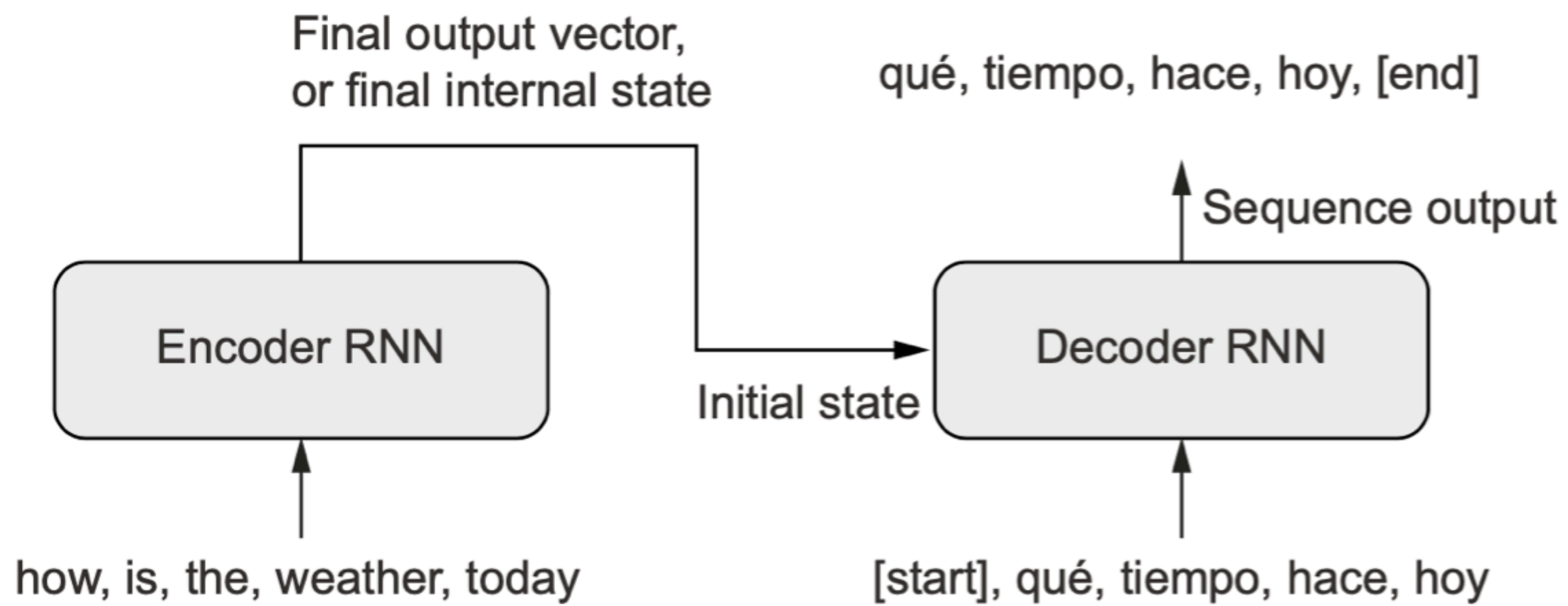
2.1 Try 1 for Machine Translation: RNN

2.2 Try 2 for Machine Translation: Transformer



Get dataset

English-to-Spanish translation dataset at www.manythings.org/anki/.



Listing 11.28 GRU-based encoder

```
from tensorflow import keras
from tensorflow.keras import layers
```

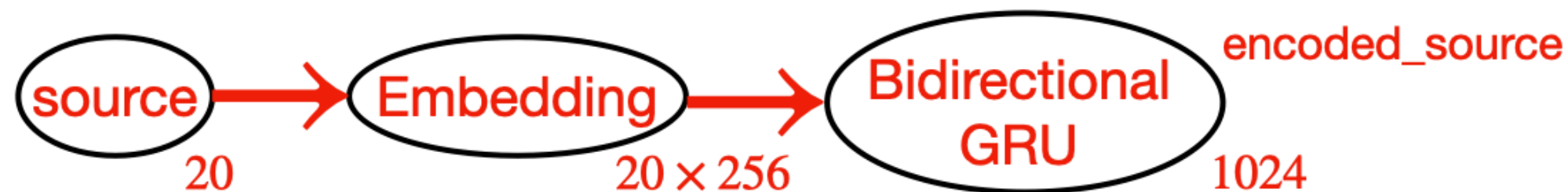
```
embed_dim = 256
latent_dim = 1024
```

```
source = keras.Input(shape=(None,), dtype="int64", name="english")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(source)
encoded_source = layers.Bidirectional(
    layers.GRU(latent_dim), merge_mode="sum")(x)
```

Don't forget masking:
it's critical in this setup.

The English source sentence goes here.
Specifying the name of the input enables
us to fit() the model with a dict of inputs.

Our encoded source
sentence is the last output
of a bidirectional GRU.



Next, let's add the decoder—a simple GRU layer that takes as its initial state the encoded source sentence. On top of it, we add a Dense layer that produces for each output step a probability distribution over the Spanish vocabulary.

Listing 11.29 GRU-based decoder and the end-to-end model

The Spanish target sentence goes here.

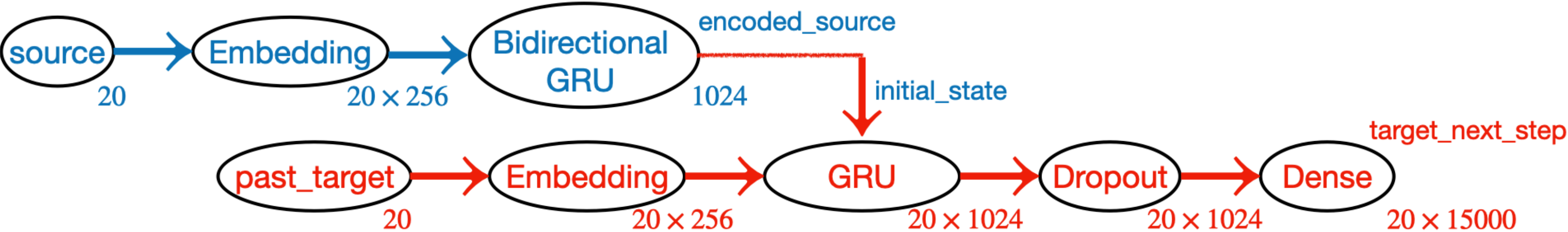
Don't forget masking.

```
past_target = keras.Input(shape=(None,), dtype="int64", name="spanish")
x = layers.Embedding(vocab_size, embed_dim, mask_zero=True)(past_target)
decoder_gru = layers.GRU(latent_dim, return_sequences=True)
x = decoder_gru(x, initial_state=encoded_source)
x = layers.Dropout(0.5)(x)
target_next_step = layers.Dense(vocab_size, activation="softmax")(x)
seq2seq_rnn = keras.Model([source, past_target], target_next_step)
```

Predicts the next token

The encoded source sentence serves as the initial state of the decoder GRU.

End-to-end model: maps the source sentence and the target sentence to the target sentence one step in the future



Ensure Causality

During training, the decoder takes as input the entire target sequence.

But thanks to the step-by-step nature of RNN, it only looks at tokens 0, 1, ..., N in the input to predict token N+1 in the output.

Listing 11.31 Translating new sentences with our RNN encoder and decoder

Prepare a dict to convert token index predictions to string tokens.

```
import numpy as np
spa_vocab = target_vectorization.get_vocabulary()
spa_index_lookup = dict(zip(range(len(spa_vocab)), spa_vocab))
max_decoded_sentence_length = 20

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

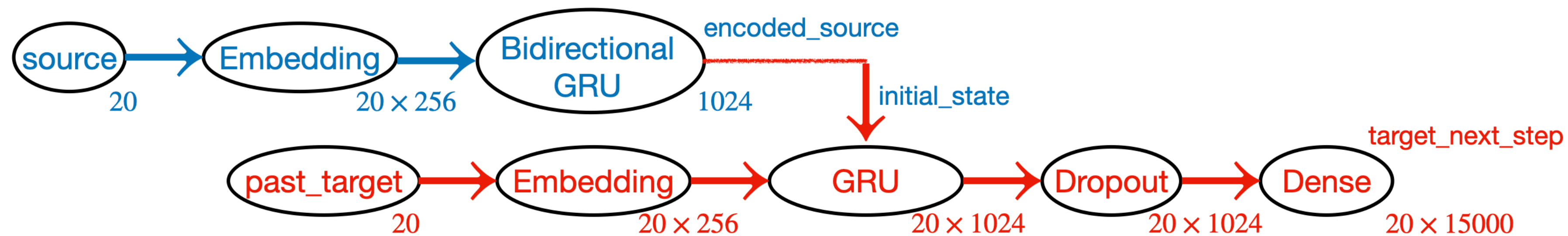
test_eng_texts = [pair[0] for pair in test_pairs]
for _ in range(20):
    input_sentence = random.choice(test_eng_texts)
    print("-")
    print(input_sentence)
    print(decode_sequence(input_sentence))
```

Seed token

Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

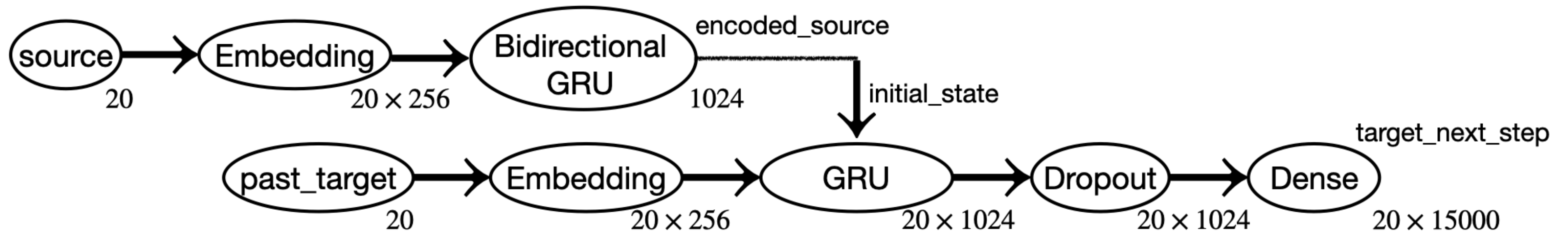
```

Seed token →

Sample the next token.

Exit condition:
either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

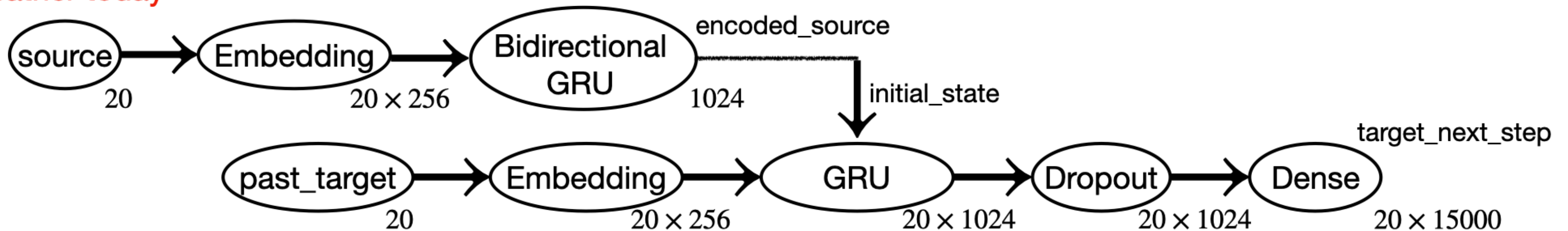
Seed token →

Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

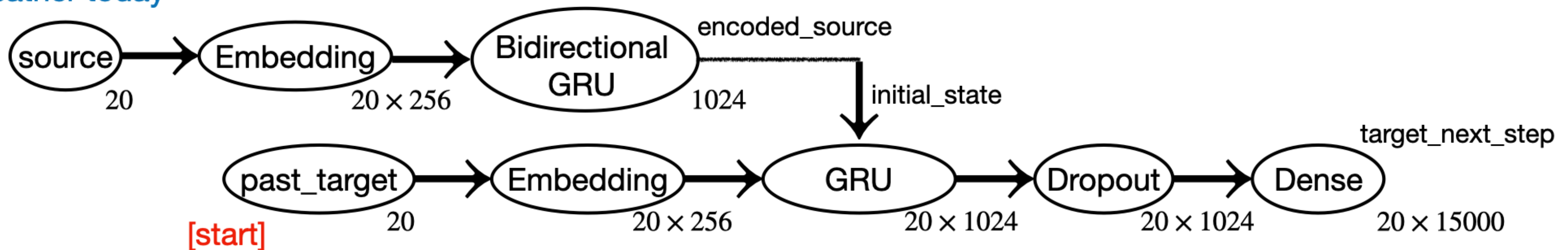
Seed token →

Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

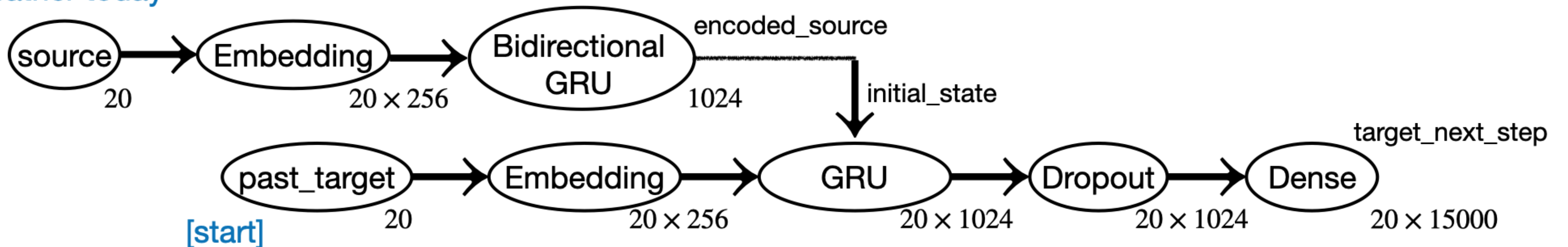
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 0$

how is the
weather today




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

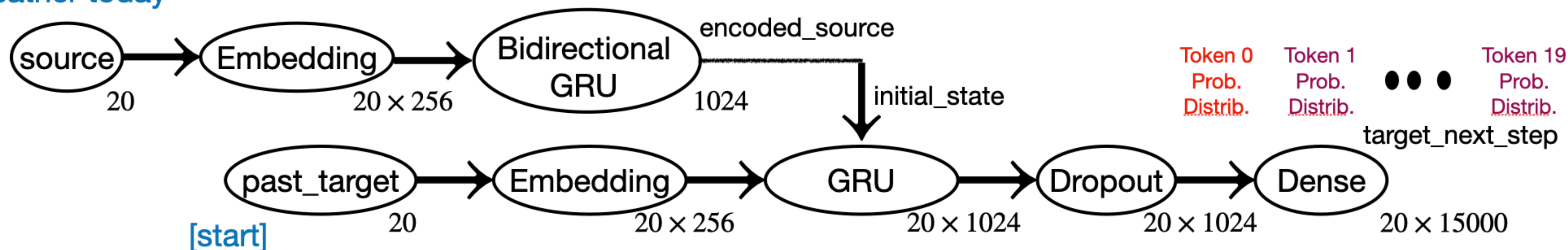
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 0$

how is the
weather today



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

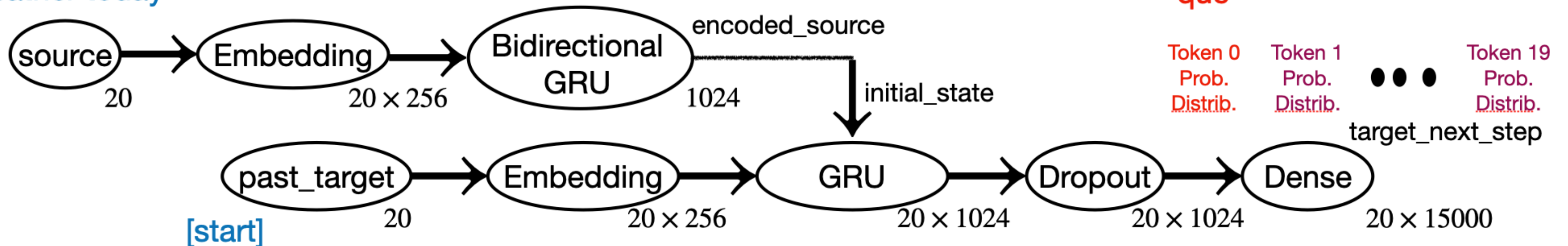
Sample the next token.

Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

how is the
weather today

$i = 0$




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token.

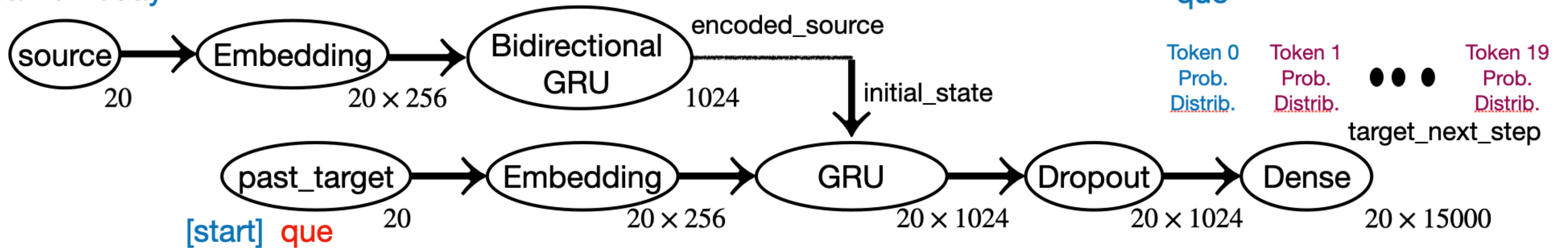
Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

how is the
weather today

$i = 0$

que



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

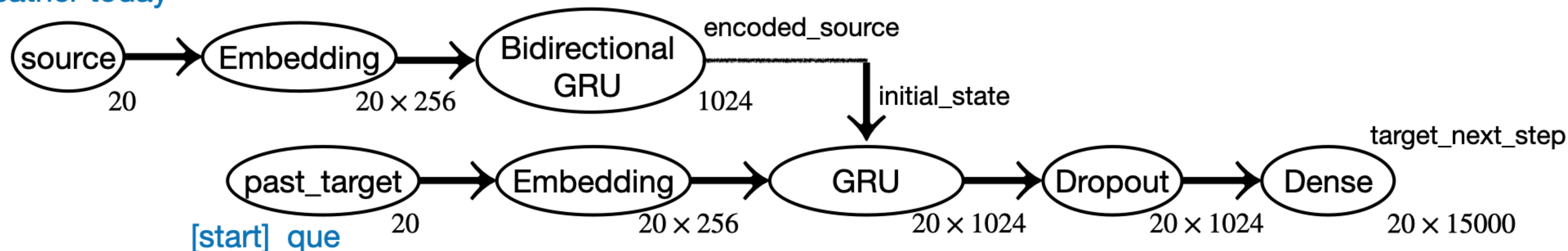
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 1$

how is the
weather today




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

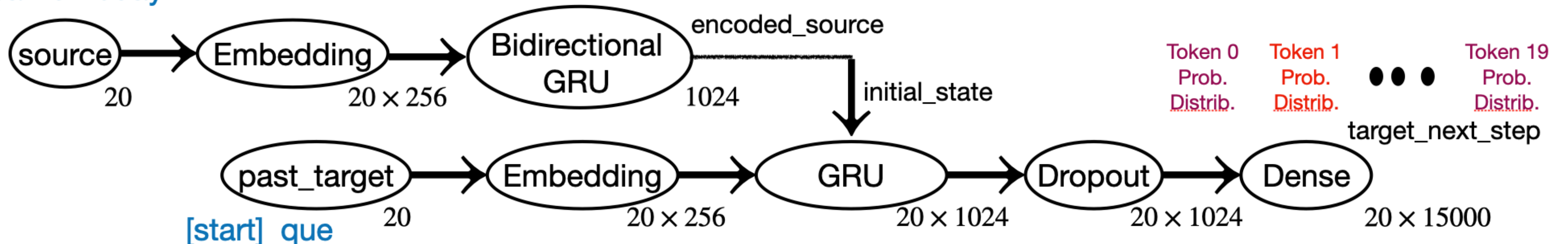
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 1$



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token. |

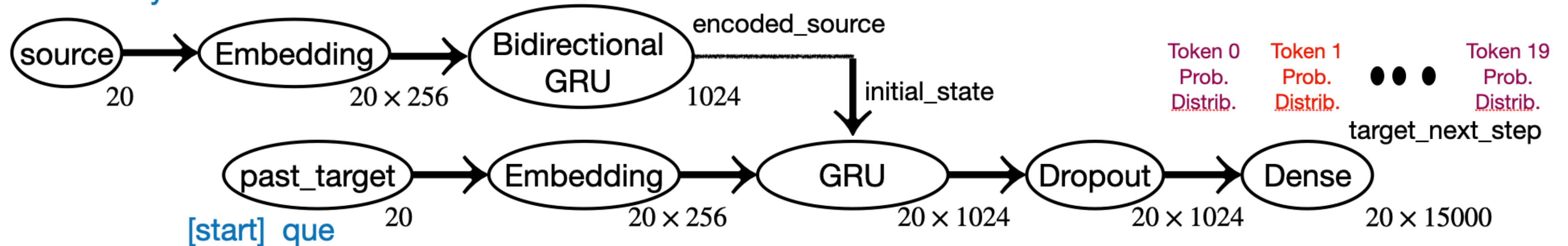
Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

how is the
weather today

$i = 1$

tiempo




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

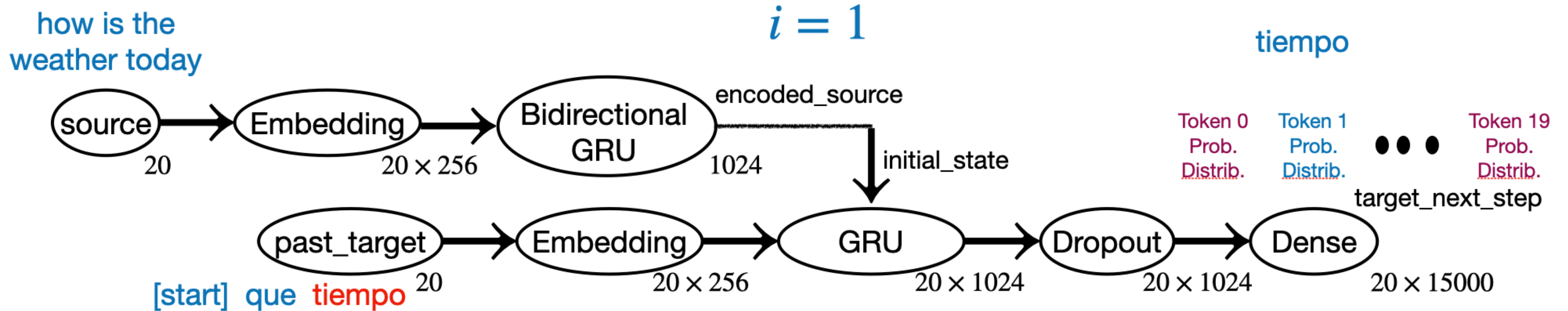
```

Seed token →

Sample the next token.

Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

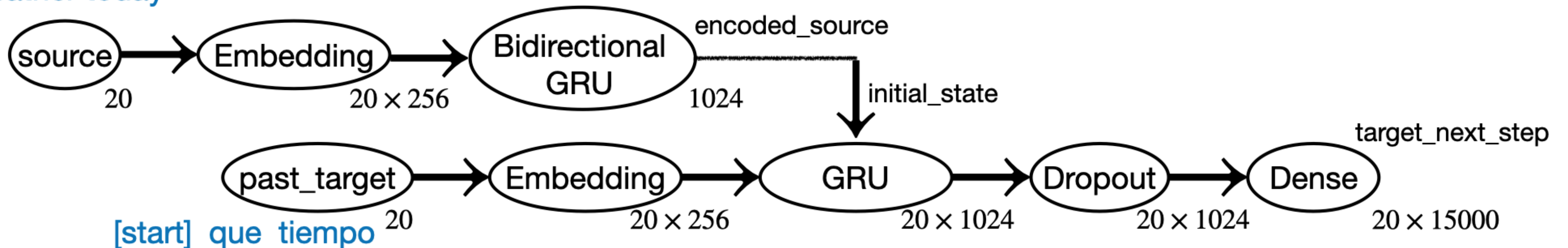
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 2$

how is the
weather today




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

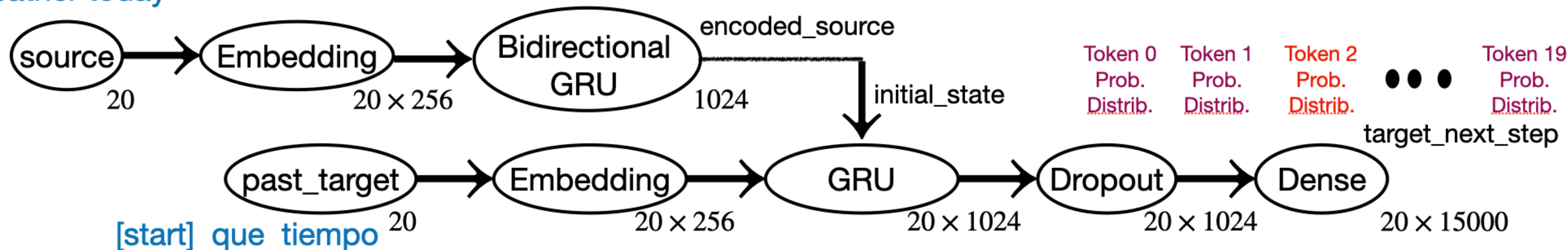
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 2$

how is the
weather today



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token.

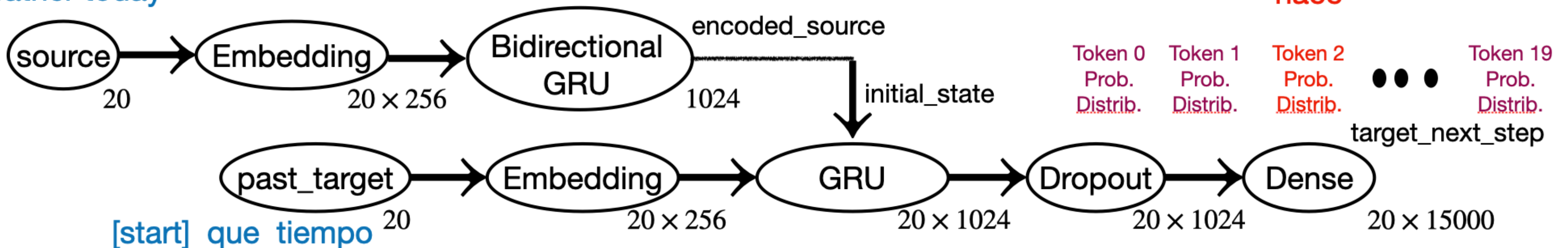
Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

$i = 2$

how is the
weather today

hace




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token.

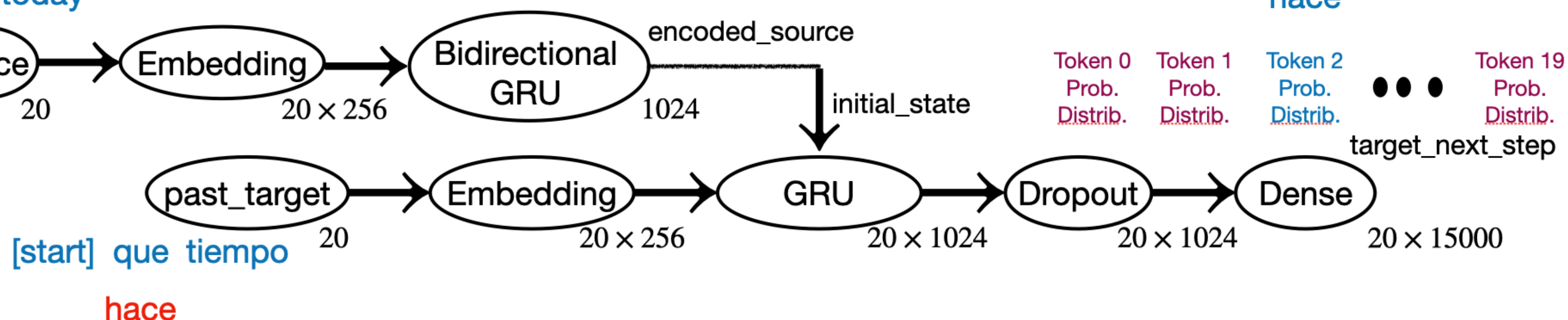
Exit condition:
either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 2$

hace



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

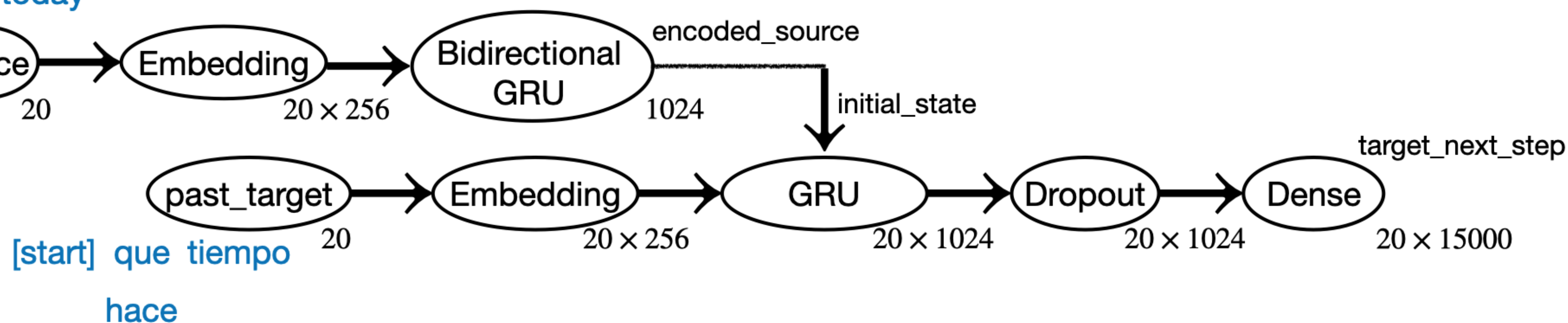
Sample the next token.

Exit condition:
either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 3$




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

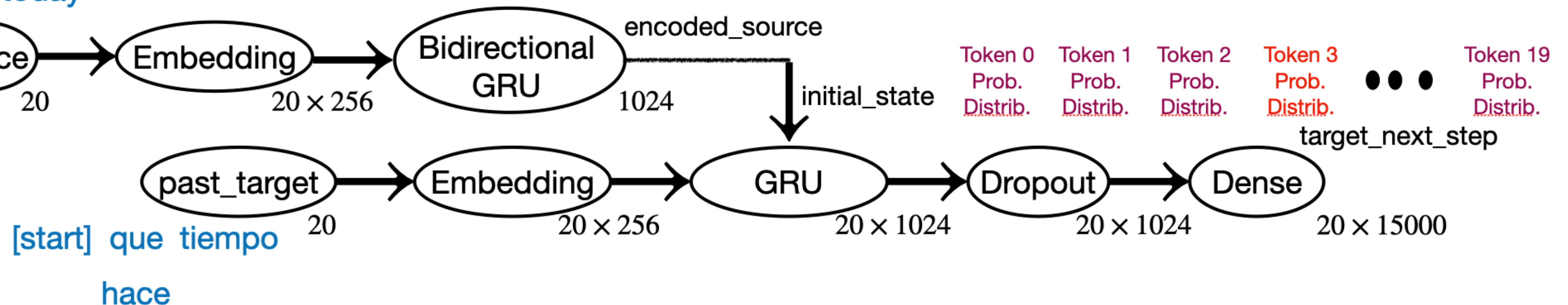
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 3$



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

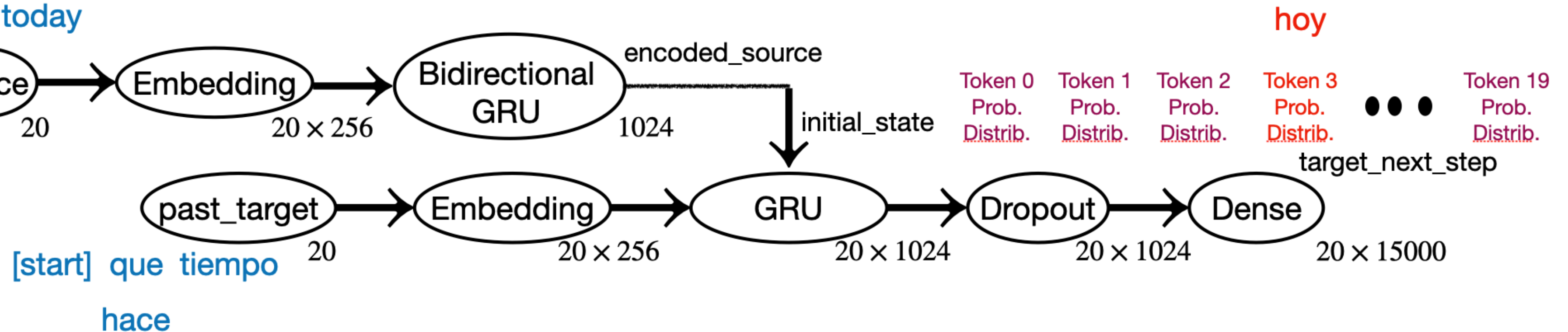
Sample the next token. |

Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

how is the
weather today

$i = 3$




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

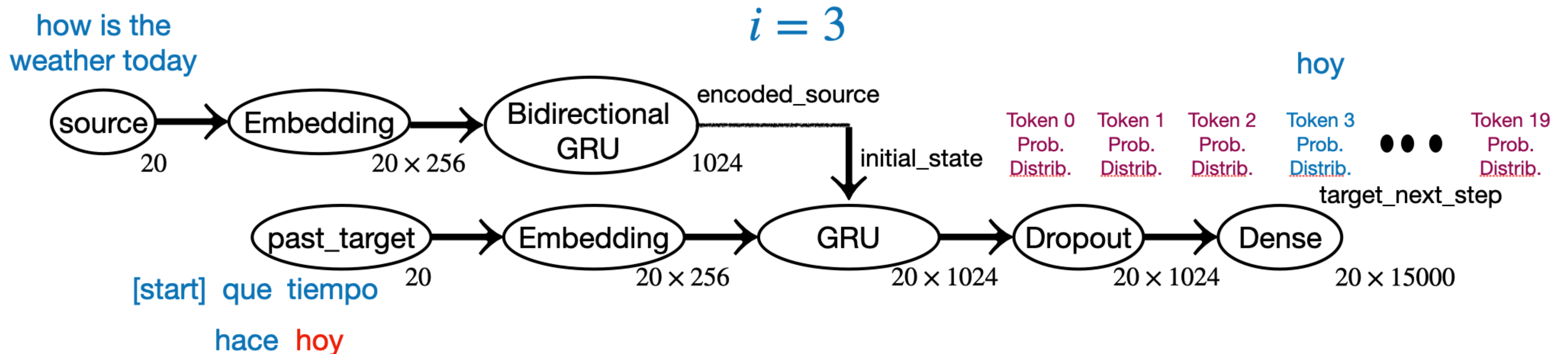
```

Seed token →

Sample the next token.

Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

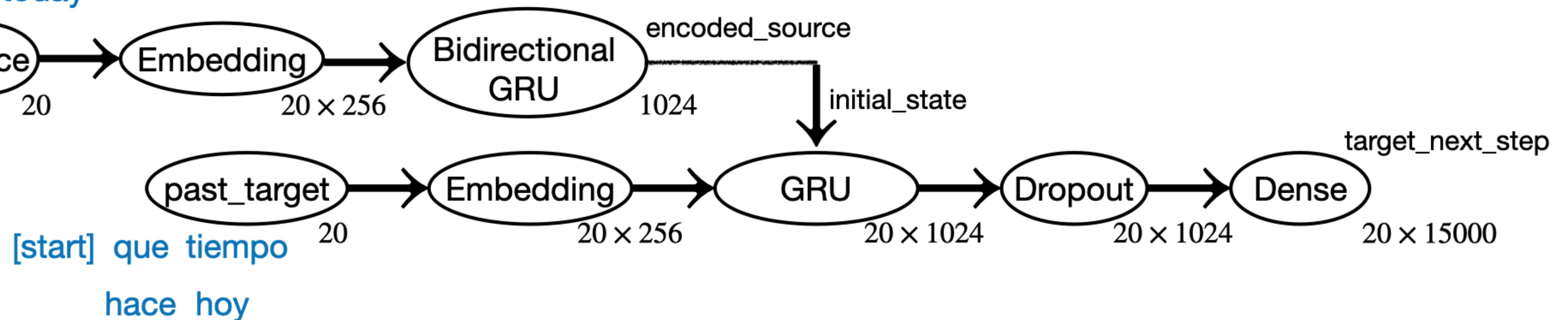
Sample the next token.

Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 4$




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

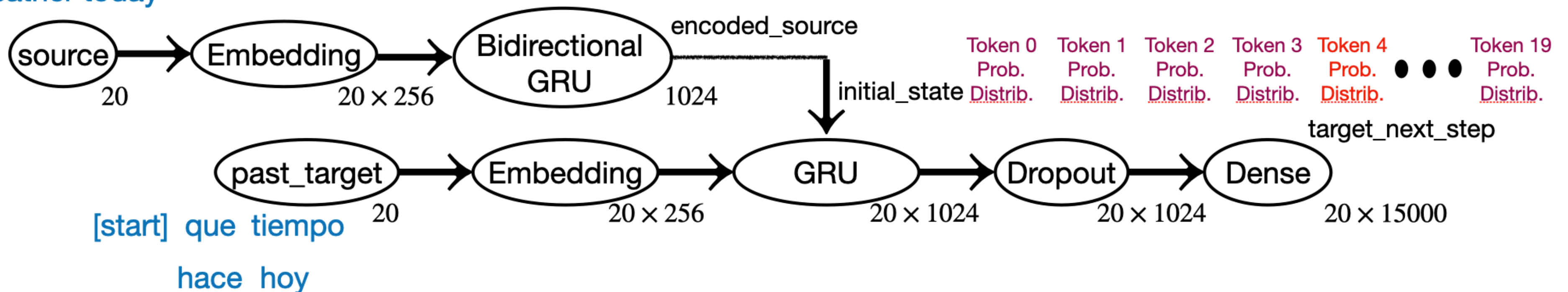
Sample the next token. |

Exit condition:
either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

$i = 4$

how is the
weather today



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token. |

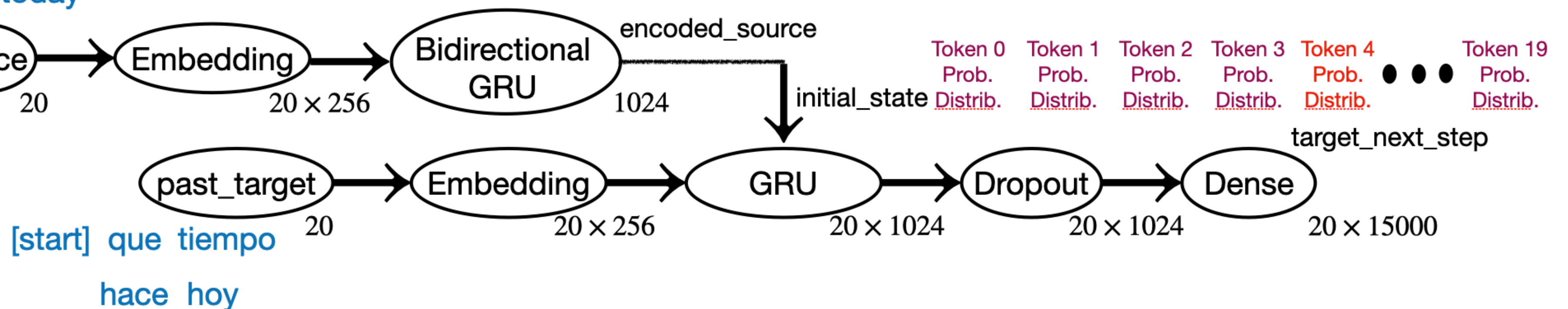
Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character

how is the
weather today

$i = 4$

[end]




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

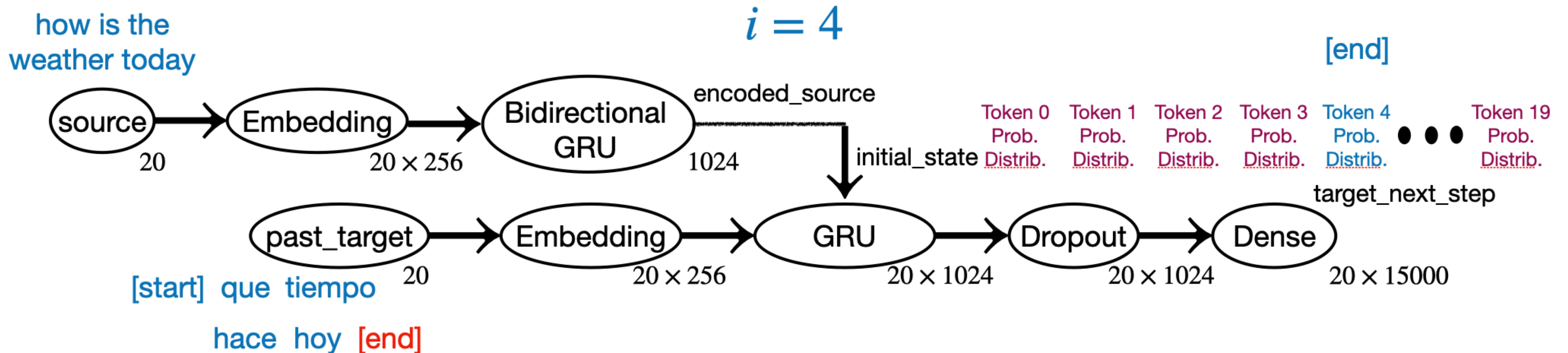
```

Seed token →

Sample the next token.

Convert the next token prediction to a string and append it to the generated sentence.

Exit condition: either hit max length or sample a stop character




```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

Seed token →

Sample the next token.

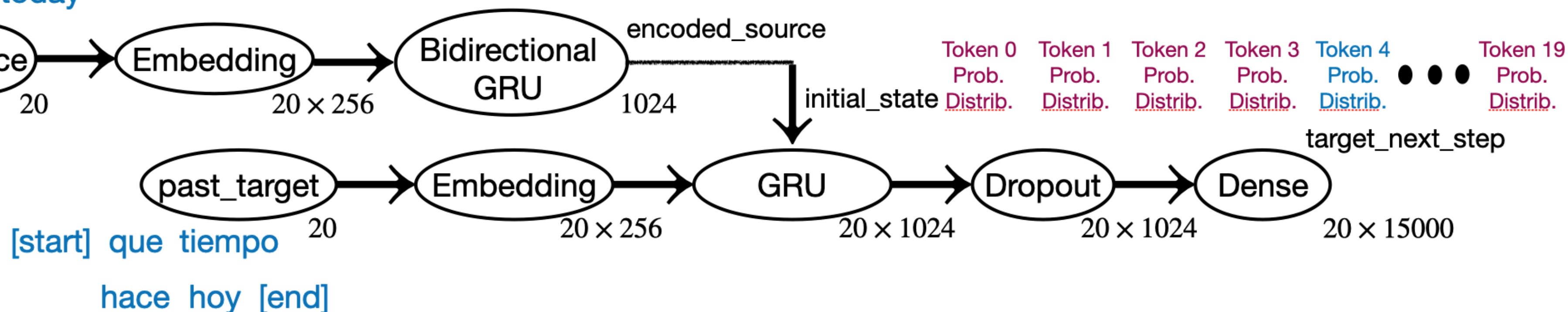
Exit condition: either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the
weather today

$i = 4$

[end]



```

def decode_sequence(input_sentence):
    tokenized_input_sentence = source_vectorization([input_sentence])
    decoded_sentence = "[start]"
    for i in range(max_decoded_sentence_length):
        tokenized_target_sentence = target_vectorization([decoded_sentence])
        next_token_predictions = seq2seq_rnn.predict(
            [tokenized_input_sentence, tokenized_target_sentence])
        sampled_token_index = np.argmax(next_token_predictions[0, i, :])
        sampled_token = spa_index_lookup[sampled_token_index]
        decoded_sentence += " " + sampled_token
        if sampled_token == "[end]":
            break
    return decoded_sentence

```

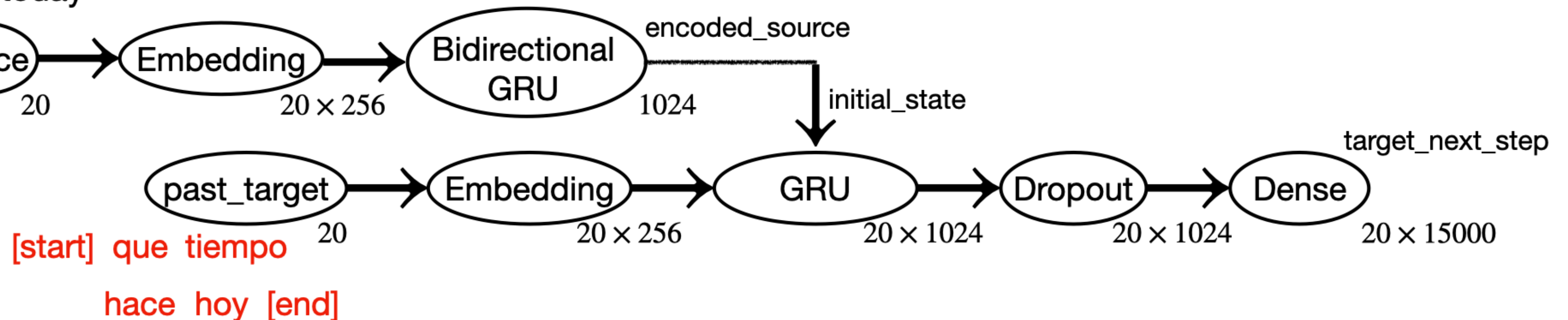
Seed token →

Sample the next token. |

Exit condition:
either hit max length or sample a stop character

Convert the next token prediction to a string and append it to the generated sentence.

how is the weather today



Quiz questions:

1. How does machine translation using RNN work?
2. What are possible ways to improve its performance?

Roadmap of this lecture:

1. Text classification for IMDB

2. Sequence-to-Sequence Learning for Machine Translation

2.1 Try 1 for Machine Translation: RNN

2.2 Try 2 for Machine Translation: Transformer



Transformer Encoder and Decoder

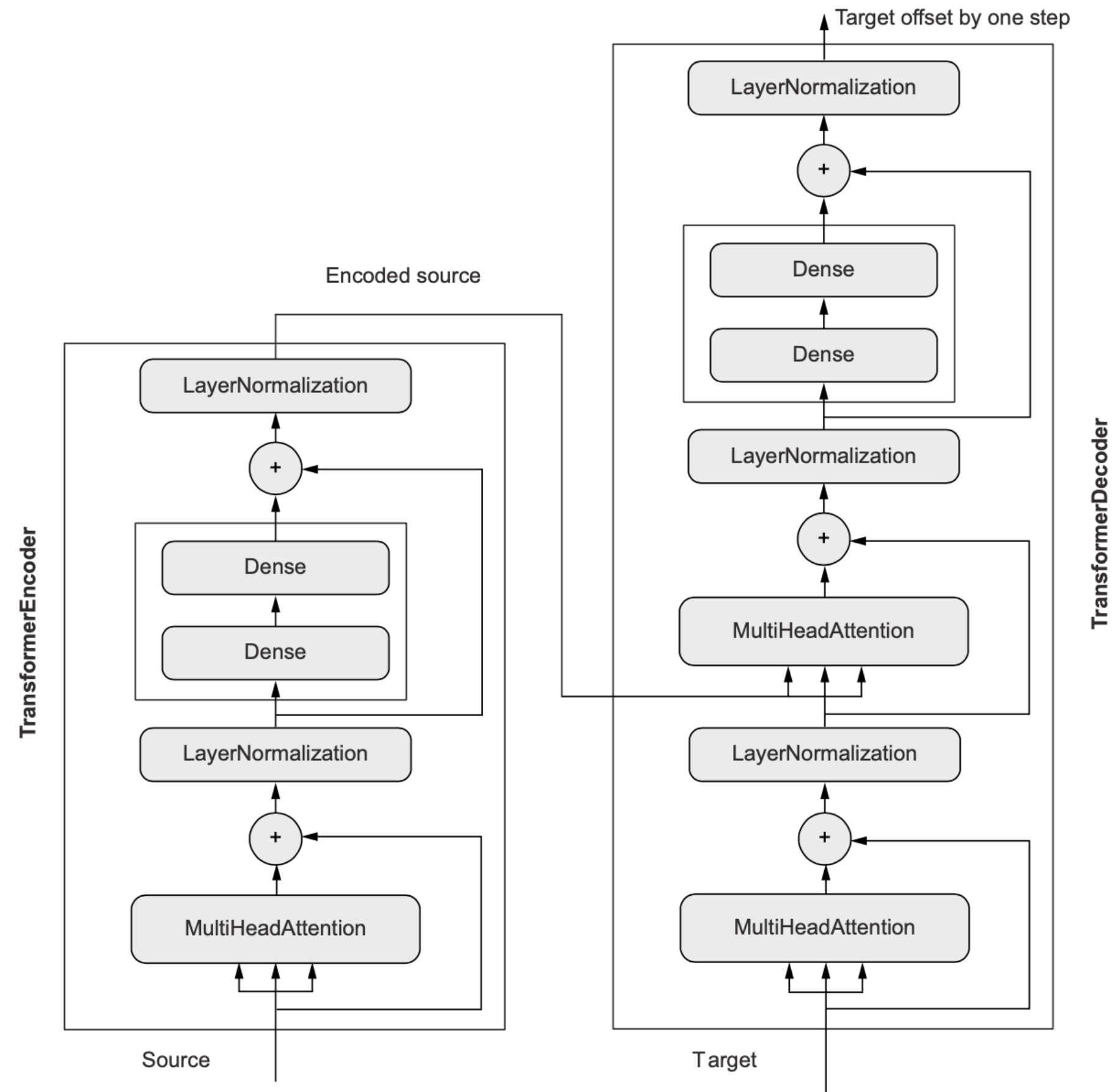


Figure 11.14 The **TransformerDecoder** is similar to the **TransformerEncoder**, except it features an additional attention block where the keys and values are the source sequence encoded by the **TransformerEncoder**. Together, the encoder and the decoder form an end-to-end Transformer.

Causal Padding for Training

During training, the decoder cannot use target words that will appear in the future.

Solution: mask the upper half of the pairwise attention matrix to prevent the model from paying any attention to information from the future. (Only information from tokens $0, 1, \dots, N$ in the target sequence are used when generating the token $N+1$.)

Listing 11.36 End-to-end Transformer

```
embed_dim = 256
dense_dim = 2048
num_heads = 8
```

Encode the
source sentence.

```
encoder_inputs = keras.Input(shape=(None,), dtype="int64", name="english")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(encoder_inputs)
encoder_outputs = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
```

```
decoder_inputs = keras.Input(shape=(None,), dtype="int64", name="spanish")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(decoder_inputs)
x = TransformerDecoder(embed_dim, dense_dim, num_heads)(x, encoder_outputs)
x = layers.Dropout(0.5)(x)
```

Encode the target sentence and combine
it with the encoded source sentence.

```
decoder_outputs = layers.Dense(vocab_size, activation="softmax")(x)
transformer = keras.Model([encoder_inputs, decoder_inputs], decoder_outputs)
```

Predict a word for each output position.

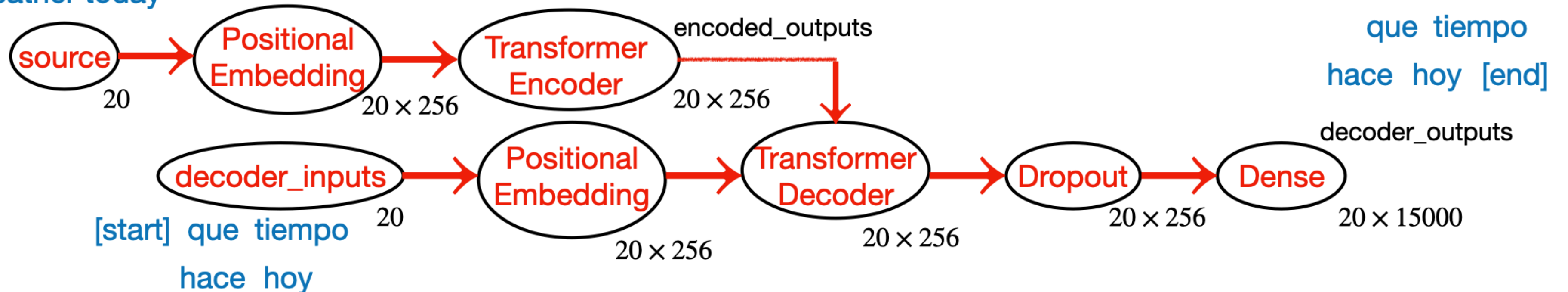
how is the
weather today

que tiempo
hace hoy [end]

[start] que tiempo
hace hoy

decoder_outputs

20×15000



Quiz questions:

1. What is the architecture of a Transformer decoder?
2. How does machine translation using Transformer work?